

The CodeBound: A 2D Game Journey Through Logic and Code Challenges

The CodeBound: A 2D Game Journey Through Logic and Code Challenges!
Presented to
The Faculty of the College of Computer Studies
Tarlac State University
Tarlac City

In Partial Fulfillment
of the Requirements for the Degree
<Degree Obtained>

by:

Elaika Joy Santiago
Eowyn Feona F. Mabanglo
Mark Guiller A. Melchor
John Lester P. Tiglao

August 1

TABLE OF CONTENTS

TABLE OF CONTENTS	2
LIST OF TABLES	5
Table 14. Sprint 4: Story Integration, Feedback, Testing 54.....	6
LIST OF FIGURES	7
1 INTRODUCTION.....	9
1.1 Project Context.....	9
1.2 Purpose And Descriptions	4
1.3 Objectives.....	5
1.4 Scope and Delimitations.....	6
2 RELATED LITERATURE.....	9
2.1 Discussion of Models	9
2.2 Conceptual Framework	24
3 TECHNICAL BACKGROUND.....	26
3.1 Software Development Requirements	26
3.2 Hardware Development.....	29

3.3	Peopleware	30
3.4	Source of Data.....	31
4	METHODOLOGY	33
4.1	Methods in Data Gathering.....	33
4.1.1	Internet Method	33
4.1.2	Observation.....	34
4.2	Design and Development of <Title> This is objective number 1.....	36
4.2.1	Requirement Analysis and Documentation	35
4.2.1.2	User Characteristics	36
4.2.1.3	Functional Requirements	37
4.2.1.4	Non-Functional Requirements.....	39
4.2.2	Design of Software and/or System and/or Product and/or Processes.....	40
4.2.2.1	ERD Diagram.....	42
4.2.2.2	Functional Decomposition Diagram	41
4.2.2.3	Data Storage Architecture	42
4.2.2.4	Operating Environment.....	43
4.2.2.5	Design and Implementation Constraints.....	44
4.2.3	System Development Methodology	45
4.2.3.1	Planning	49
4.2.3.2	Project Schedule	55
4.3	To evaluate the performance of CodeBound: A 2D Game Journey Through Logic and Code Challenges.....	56
4.3.1	Testing.....	56
4.3.2	Evaluation.....	66

4.4	Storyboard	68
-----	------------------	----

REFERENCES.....	87
-----------------	----

LIST OF TABLES

Table 1. Functionality and Feature Matrix.....	13
Table 2. Literature Matrix	14
Table 3. Software Development Requirements.....	32
Table 4. Hardware Development Requirements	33
Table 5. Peopleware	34
Table 6. Functional Requirement for User Module	39
Table 7. Non-Functional Requirement for Safety and Security of The CodeBound: A 2D Game Journey Through Logic and Code Challenges!	40
Table 8. Operating Environment	45
Table 9. Design and Implementation Constraints	46
Table 10. Product Backlog.....	49
Table 11. Sprint 1: UI and 3D Asset Design	52
Table 12. Sprint 2: Game Environment & Puzzle Logic.....	53
Table 13. Sprint 3: Level Expansion & Gamification	54

Table 14. Sprint 4: Story Integration, Feedback, Testing	54
---	-----------

Table 15. Sprint 5: Website, Installer & Final Polish	55
--	-----------

LIST OF FIGURES

Figure 1. Conceptual Framework of CodeBound

Figure 2. Use Case Diagram

Figure 3. ERD Diagram

Figure 4. Functional Decomposition Diagram

Figure 5. Agile Scrum Methodology

Figure 6. Gantt chart

Figure 7. Main Menu

Figure 8. Options

Figure 9. Character

Figure 10. Save File Creation

Figure 11. Shop

Figure 12. Achievements

Figure 13. Level selection

Figure 14. Instructional interface

Figure 15. duplication Mechanic

Figure 16. Tokens

Figure 17. Boolean Mechanic

Figure 18. In Game Terminal

Figure 19. Website's Home Page

Figure 20. Website's trailer Page

Figure 21. FAQ's Page

Figure 22. Reviews Page

Figure 23. Community Hub Page

Figure 24. Website's Home Page(Mobile)

Figure 25. Website's Trailer Page (Mobile)

Figure 26. FAQ's Page (Mobile)

Figure 27. Review's Page (Mobile)

Figure 28. Community Hub Page (Mobile)

1 INTRODUCTION

1.1 Project Context

Programming has become an essential skill in today's digital world[1]. Cheah[2] notes that many students, even those pursuing computer-related courses, continue to struggle with fundamental programming concepts. These difficulties are commonly linked to several factors, such as inadequate teaching strategies, weak problem-solving abilities, and misunderstandings about programming principles[3]. The literature also highlights challenges students face during both the problem-solving and implementation stages, including the translation of logical thinking into actual code and the debugging process. Conventional teaching methods that focus heavily on syntax rather than conceptual understanding often fail to deliver meaningful and practical learning experiences. To address these issues, Cheah emphasizes the need for more engaging, interactive, and problem-solving-oriented teaching techniques that can encourage critical thinking and real-world application.

One promising approach to addressing the challenges in programming education is the use of game-based and puzzle-based learning strategies[4]. These methods offer an engaging alternative to traditional lecture-based instruction by immersing students in interactive problem-solving scenarios[5]. Puzzle-based learning, in particular, enhances student engagement and retention by creating a fun and interactive environment. Findings show that most students find puzzle activities both enjoyable and effective, with 21 out of

27 students reporting increased participation, and all agreeing that puzzles should be used in lessons[6]. When applied to programming, puzzles and game-like elements can reduce the intimidation often associated with learning to code, while boosting motivation, critical thinking, and conceptual understanding[5]. These strategies have the potential to improve learning outcomes not only in language learning but across various disciplines, including computer science[6].

One of the key benefits of educational game-based learning in early childhood education is its ability to promote cognitive development. According to Nazilah et al.[7], online games can help children enhance their problem-solving skills, memory, attention, and processing speed. For example, puzzle games improve spatial reasoning, while memory games enhance concentration and recall. Online games can also support emotional development in children. As noted by Alotaibi[4], such games help enhance emotional regulation, self-awareness, and self-confidence.

Gamification in education helps reinforce concepts by offering instant feedback, interactive challenges, and repetition in a fun, engaging format. It supports constructivist learning theories, where learners actively construct knowledge through experience.[22] Gamification is defined as the application of game design elements and principles in non-game contexts to enhance user engagement, motivation, and behavior.[25] Gamification is justified as an effective tool for influencing positive behavioral change because it leverages motivational design to encourage participation, persistence, and engagement.[23] Game mechanics like points, levels, and leaderboards provide users with constant feedback, which enhances self-assessment and promotes continuous improvement.[24] In organizational settings, gamification enhances employee morale, fosters healthy competition, and increases productivity.[22][23] These outcomes are supported by empirical research showing that gamified systems lead to higher task

engagement and improved performance in both educational and professional environments.[23]

In this paper, the researchers aim to develop a 2D puzzle-based game designed to help students acquire programming skills in an engaging and enjoyable way. A puzzle game is a type of video game that challenges players to solve problems using logic, pattern recognition, sequence solving, spatial reasoning, or strategy. In *CodeBound*, these puzzle elements are integrated into both environmental interactions and Java-based coding challenges as the player, a student trapped in a mysterious digital world, must ascend 100 floors to return home. Players manipulate objects, unlock doors, and solve clues by writing code, identifying patterns, executing actions in the correct order, and strategically coordinating multiple characters. By merging traditional puzzle mechanics with interactive programming, the game promotes logical thinking, problem-solving, and creativity, making coding more accessible and enjoyable.

1.2 Purpose And Descriptions

The purpose of this study was to develop CodeBound: A 2D Game Journey Through Logic and Code Challenges!, an educational game designed to strengthen students' foundational programming skills through interactive, puzzle-based learning. The game aimed to support programming education by fostering logical thinking, problem-solving, and creativity in a more engaging and motivating format.

Rather than replacing traditional instruction, CodeBound was intended to complement classroom teaching by serving as an alternative learning and assessment tool. It introduced students to core programming concepts through a structured series of logic puzzles set in a 2D environment. The puzzles progressed from basic to intermediate difficulty, covering essential Java programming topics such as variables, data types, loops, conditionals, and functions.

The game was specifically designed for first-year students or beginner programmers. A gradual increase in difficulty ensured that learners were appropriately challenged as they developed their skills, with built-in guidance and hints to provide support and prevent frustration.

By integrating gameplay with educational content, CodeBound transformed traditional coding exercises into interactive challenges, allowing students to apply theoretical knowledge in a practical and enjoyable way. The use of Java as the primary programming language allowed for focused, in-depth learning aligned with introductory programming curricula.

1.3 Objectives

The general objective of this project is to design and develop a puzzle-based game with the integration of 2D environment. The game provides an engaging and immersive learning experience for the students to delve into navigate and solve programming related puzzles to enhance their fundamental programming skills.

Specifically, the project aims to achieve the following:

1. To design and develop CodeBound and implement a web application with the following features:

1.1 Core Gameplay

1.1.1 Puzzle Solving

1.1.2 Level-Based Objectives

1.1.3 Code-Based Interactions

1.1.4 Progressive Difficulty

1.1.5 2D Interactive Environment

1.1.6 Sole Programming Language Support (Java)

1.1.9 Gamification and Customization

1.1.9.1 Tokens

1.1.9.2 Achievements

1.1.9.3 Character Skins

1.2 Basic Overview of the System

1.3 Download link

1.4 Community Hub

1.5 Game Trailer

1.6 FAQs

2. To test and evaluate CodeBound User Experience and Learning features in terms of the following:

- 2.1 Testing

- 2.1.1 Black Box Testing

- 2.2 Evaluation

- 2.2.1 Functionality

- 2.2.2 Performance

- 2.2.3 Usability

1.4 Scope and Delimitations

The game was designed as a single-player, progressive learning experience and was developed using Unity, a game engine that supports the creation of 2D educational games. Unlike traditional approaches that relied on lectures or written exercises, this game immersed players in a 2D environment where they solved programming challenges using Java, the sole programming language implemented in the system. The game was made available for download through a dedicated website and supported both PC and Android platforms.

The system followed a structured algorithm that guided players through 100 levels, each representing a unique programming task. Upon launching the game, assets, the user interface, and player data were initialized before a level-specific challenge was presented. Players were required to input Java code, which was then parsed, executed, and evaluated against predefined test cases. If the solution was incorrect, the game provided detailed error messages, allowing players to revise and retry. Completion of a level unlocked the next, with challenges becoming increasingly complex to support skill development.

The game included a pause and auto-save feature, enabling players to continue at their own pace. While the game was progressive, it was not limited to a single session, allowing players to pause and resume as needed. Immediate level progress and session data were stored locally (in JSON files). However, critical user data, including profiles, achievements, and tokens, was synchronized via the Firebase cloud database. This hybrid approach ensures cloud backup for player milestones and enables cross-device syncing of high-level progress.

A clear progression system was implemented, beginning with foundational concepts in early levels such as variables, loops, and conditionals. Mid-level stages introduced functions, nested loops, and basic debugging, while the final stages covered more advanced Java concepts such as method structures and logical problem-solving. This gradual difficulty curve ensured that learners-built confidence and competence step by step.

Gamification elements were integrated to enhance engagement. Players earned badges for key achievements, such as completing specific types of challenges, and collected tokens by finishing levels. These tokens could be spent on unlocking hints, character customization options, and coding themes, adding an element of personalization and reward to the learning process.

Regex (Regular Expressions) was utilized in the in-game terminal to provide intelligent feedback. When players entered a line of Java code, the game applied regex patterns to check if the input matched the correct syntax. If it did not, the system identified the error type—such as missing punctuation or incorrect structure and provided hints like

“Did you forget a semicolon?” or “Invalid variable name format.” This enhanced the player’s understanding by offering educational, real-time error correction.

To support academic use, a teacher-access feature was developed. This allowed instructors to monitor individual student progress, including levels completed, time spent on each challenge, and common mistakes. A simple dashboard was provided to export performance summaries, giving educators a way to assess student learning informally. Additionally, teacher training materials were included to help instructors integrate the game effectively into their classroom activities.

The CodeBound website served as a promotional and informational platform. It featured sections such as Home, Trailer, FAQ, Reviews, and Community, all designed to introduce users to the game’s purpose, benefits, and features. The site included user authentication, requiring signup and login to access community interactions. Built with a modern and responsive design, the website adapted to various screen sizes and devices. It did not contain the actual game nor offer e-commerce or monetization functions. The website was available in English only, as localization was not included within the project’s scope. These constraints ensured that the project remained feasible within the team’s timeline and available resources.

2 RELATED LITERATURE

2.1 Discussion of Models

The execution model of CodeBound: A 2D Game Journey Through Logic and Code Challenges! was founded on a structured set of algorithms that governed the game’s functionality, user interaction, learning mechanics, and progression system. These algorithms defined how the game operated internally—from managing gameplay states to evaluating code and rewarding performance. Designed and implemented using the Agile–Scrum methodology, the model ensured an iterative, user-centered approach where each system component was continuously refined and aligned with the game’s educational objectives.

At the foundation of the game’s control logic was the **Finite State Machine (FSM)** algorithm. This model defined all major gameplay states such as the main menu, profile, save file selection, code terminal, feedback interface, shop, and level completion screen. Transitions between these states were dictated by user inputs and system responses. For example, when a player selected “Play” from the main menu, the FSM transitioned the system to the save file selection state, and upon code submission, it moved to either a feedback or level completion state depending on the outcome. FSM ensured that these transitions followed a predictable, stable, and error-free flow, providing players with a seamless and consistent experience.

The educational core of **CodeBound** was the **in-game terminal**, where players engaged with programming challenges by writing Java code. The terminal utilized a **Regular Expression (Regex)–based algorithm** to parse and analyze the syntax of the

submitted code. This allowed the system to detect and highlight common syntax errors such as missing punctuation, misused language structures, and incorrect variable names. Immediate and descriptive feedback was delivered through the output panel, allowing learners to identify mistakes, make revisions, and improve debugging skills. This real-time syntax validation promoted an active, trial-and-error learning approach fundamental to programming education.

Following syntax validation, the game employed a **Test Case Validation Algorithm** to determine whether the submitted code's logic met the requirements of the challenge. Each level contained predefined test cases with expected input–output pairs. When players ran their code, the system executed it and compared the results to the expected outputs. A full match across all test cases indicated level completion, while mismatches prompted feedback and allowed retries. This algorithm simulated a **unit testing** environment, reinforcing computational thinking and logical problem-solving under real coding conditions.

To structure the player's learning path, **Sequential Progression Logic** was implemented. Players were required to complete levels in order, progressing from basic to more advanced programming concepts across one hundred challenges. This linear system prevented skipping and ensured that foundational skills were built before introducing complex topics. The progression logic also worked with the **local data storage system**, which recorded user progress, customization preferences, and achievements using JSON formatting. This feature enabled players to resume gameplay seamlessly without requiring an internet connection.

To sustain user motivation, the game incorporated **Gamification and Reward Logic**. As players completed levels, they earned tokens and unlocked achievements based

on their performance—considering factors such as completion time, number of retries, and hint usage. Tokens could be spent in the in-game shop to unlock customization options, while achievements provided visible recognition of milestones. These elements were managed by an algorithm that tracked user activity, dynamically updated rewards, and promoted engagement and replayability.

All of these algorithmic components were developed using the **Agile–Scrum framework**, allowing features to be designed, tested, and refined in iterative sprints. Each sprint focused on distinct modules, including UI development, gameplay logic, database integration, and web connectivity. Regular sprint reviews and retrospectives ensured that the team adapted quickly to user feedback and maintained alignment with educational and technical objectives.

Overall, the **execution model of CodeBound** represented a cohesive integration of algorithms—finite state logic for flow control, regex-based syntax analysis, test case validation for logical accuracy, sequential progression for structured learning, and gamified feedback for engagement. Developed under Agile–Scrum, this model ensured that CodeBound functioned not only as a technically stable 2D game but also as an effective educational platform for enhancing beginner-level programming skills.

Table 1. Functionality and Feature Matrix

Software	1	2	3	4	5
CodeCombat	✓	✓	✓	✓	✗
Human Resource Management	✓	✓	✗	✗	✗
Scratch	✗	✗	✓	✗	✗
CodinGame	✓	✓	✗	✗	✗
Minecraft Education	✓	✗	✓	✗	✗

1. Code execution
2. Puzzle-Based
3. Tutorial/Guide
4. Story mode
5. Curriculum synced

Table 1 presented the comparison of five existing game-based learning platforms based on their key functionalities, including CodeCombat, Human Resource Machine, Scratch, CodinGame, and Minecraft: Education Edition. Each platform was designed to teach programming but varied in instructional approach and target audience.

Code execution was a core feature in all except **Scratch**, which relied on block-based visual programming rather than text-based syntax. **Multilanguage support** also differed among the platforms: **CodinGame** supported over 25 programming languages, **CodeCombat** focused on **JavaScript** and **Python**, **Human Resource Machine** was limited to **C++**, and **Minecraft: Education Edition** allowed both **Python**, **JavaScript**, and block-based coding through MakeCode.

Tutorials and guided lessons were provided by most platforms to support beginner learners. However, **Human Resource Machine** and **CodinGame** assumed users already possessed basic programming knowledge, while **CodeCombat** and **Minecraft: Education Edition** offered structured, step-by-step learning experiences suitable for novices. Among them, **CodeCombat** was the only platform that utilized a **narrative-driven progression system**, enhancing engagement through story-based missions, whereas **Minecraft: Education Edition** emphasized **open-ended exploration** and creative problem-solving within guided lesson worlds.

Lastly, none of the reviewed platforms adhered to a **standardized curriculum**; instead, they offered **general learning paths** focused on skill development rather than formal educational alignment. Recognizing these gaps, the researchers of this study aimed

to integrate the **most effective features**—structured progression, code validation, gamification, and accessible tutorials—into the proposed system, **CodeBound: A 2D Game Journey Through Logic and Code Challenges**, to provide a more cohesive and curriculum-aligned learning experience

Table 2. Literature Matrix [LOCAL AND INTERNATIONAL]

REFERENCE	DESCRIPTION	STRENGTH	WEAKNESS
Dollente, DK. et al. (2023)[12]	"ANITO: Battle of the Gods" is an edutainment game that integrates Philippine cultural elements to create an engaging and educational experience.	The game successfully combines entertainment with education. Through interactive gameplay, players are exposed to Filipino traditions, beliefs, and mythology. This makes the game especially appealing in educational or cultural settings	The game only features four playable deities, which restricts variety and replayability. Additionally, there are no options for character customization or alternate costumes

Buenaflor (2024)[13]	A. AGILA is a game-based learning activity designed to enhance student's learning attitude towards Araling Panlipunan.	Students gain a more engaging learning experience; teachers are provided with an innovative teaching strategy	Dependent on External Developer which may create future dependency for updates, bug fixes, and localization issues.
Lagayan, E. C., Long, J., and Mojica, A. R. (2023)[14]	PortaLogica is a 2D game designed to teach logic gates in an interactive and engaging way. Developed for computer science, IT, and computer engineering students.	The game performs smoothly, using system resources efficiently without noticeable lag or unnecessary delays.	Participants noted that the lack of a sprint function made navigation feel slow and somewhat tedious.
Amolat, N. I. L., Consuelo, J. P. R., and Defacto, H. M. G. (2022)[15]	Lakbay is a 3D educational mobile game developed to teach driving fundamentals and	The game includes a language toggle between English and Filipino, making the content more	Although materials are said to be based on the Filipino Driver's Manual and LTO, there is no formal

	road safety, created for Gear-1 Driving School. It features interactive modules like quiz-based driving simulations and mission-based challenges, with support for both English and Filipino.	accessible and user-friendly for Filipino learners.	certification or endorsement from the LTO.
Batoon, J. A., Antonio, L. G., Dongs, J. P., and Fernandez, M. Y. D. Jr. (2022)[16]	This study presents the development of a 3D action role-playing game (ARPG) titled “ <i>The Light of End</i> ” that integrates basic addition problems into its gameplay.	One major advantage of the game is that it can be played without an internet connection. This makes it highly accessible for students in areas with poor or no internet access,	Mechanics is limited to basic addition problems. Expanding the mathematical content to include subtraction, multiplication or division problems would provide a broader educational experience.
Cheung SY and Ng KY (2021)[17]	The study uses <i>PaGamO</i>	The study highlights that students found	Only multiple-choice questions were used,

	educational platform that allows students to answer questions. It enhances engagement and motivation by combining learning with strategy-based mechanics.	<i>PaGamO</i> fun, engaging, and effective for self-learning or exam learning through a trial-and-error approach.	which may not support deeper learning or exam preparedness for other question types.
Tacouri, H. and Nagowah, L. (2021)[18]	Code Saga is a serious mobile game designed to help undergraduate IT students learn programming in an engaging and interactive way. The game aims to make self-learning more effective by integrating educational content with gameplay.	The game incorporates visualization-based learning approaches that help students grasp abstract programming concepts.	Students expressed a desire for more content and challenge within the game. Adding more levels, missions, and progressively complex tasks
Vahldick, A. et al. (2020)[19]	NoBug's SnackBar is a game developed to	The development of a level and mission	Initial feedback revealed that some

	help develop problem-solving skills useful in introductory programming courses.	editor empowers the educators to tailor the game to their curriculum without requiring technical expertise in XML or SQL.	students felt overwhelmed by the volume of concepts introduced in a short play session.
E. Byusa, E. Kampire, and A. R. Mwesigye (2022)[20]	systematic review that explores how game-based learning (GBL) improves students' motivation and understanding of chemistry concepts.	uses interactive simulations, visuals, and storytelling that help students better grasp complex or abstract chemistry topics	Games may shift focus from learning to fun, causing students to engage superficially without gaining deep understanding.
Demiray, B. Z., Sermet, Y., Yildirim, E., & Demir, I. (2023)[21]	Introduces Flood Game, a web-based 3D serious game designed to educate students and the general public about flood risk, mitigation strategies, and disaster preparedness.	Users can select predefined maps or create maps from real-world locations via Google Maps integration.	Although it runs in browsers, the game still requires stable internet and some computing.

Local Literature/Studies

Dollente et al. [12] developed “Anito: Battle of the Gods” to showcase Philippine mythology by blending entertainment and education. It offers an arcade-style story mode where players battle deities, unlocking character-specific endings that deepen both gameplay and cultural exploration. With captivating aesthetics and immersive storytelling, the game serves as a bridge to understanding and appreciating the rich mythology of the Philippines. Their study highlights how storytelling and interactive gameplay can enhance cultural learning. While their game relies on 2D visuals and action-based mechanics, this study explores 3D visual enhancements to improve spatial awareness and engagement in a puzzle-based coding game.

CodeBound builds on this by offering a 2D puzzle-based learning adventure with a robust narrative that guides players through a digital tower, solving programming challenges. Unlike ANITO, CodeBound aligns with computer science curricula, ensuring its puzzles reinforce academic learning outcomes. Additionally, it provides extensive character customization through unlockable skins and outfits earned via achievements and tokens, enhancing replayability and personalization.

Buenaflor[13] explores the effectiveness of AGILA, a game-based learning activity aimed at improving students' learning attitudes toward Araling Panlipunan. Conducted at Luis Hervias National High School in Bacolod City, the research reveals that AGILA significantly enhances students' attitudes toward learning, despite some engagement challenges associated with the modular approach. The findings underscore the potential of game-based learning in fostering motivation and emotional engagement among students.

To address the dependency on external developers, CodeBound is designed and maintained directly by the researchers, ensuring full control over future updates, bug fixes, and localization. Its 2D environment delivers a curriculum-synced adventure that immerses players in a storytelling-driven journey, teaching programming concepts like variables and loops through engaging puzzles. This focus on narrative and curriculum alignment distinguishes CodeBound from AGILA's broader subject focus, providing a targeted tool for computer science education.

Lagayan et al.[14] developed PortaLogica, a 2D game-based learning application focused on teaching logic gates. The game was designed for computer science, IT, and computer engineering students, aiming to provide an interactive and immersive learning experience. Players solve logic gate puzzles within a story-driven environment, reinforcing theoretical concepts through hands-on gameplay. CodeBound enhances this model by incorporating a sprint mechanic in its 2D environment, doubling character movement speed for efficient exploration. Its narrative-driven adventure, where players ascend a digital tower solving programming puzzles, is synced with academic curricula, ensuring relevance to foundational computer science concepts. This combination of storytelling, curriculum alignment, and improved mobility sets CodeBound apart from PortaLogica's logic-gate focus.

Amolat et al.[15] developed Lakbay, a three-dimensional game-based learning application focused on teaching driving fundamentals, road courtesy, and safety. The game was designed for students of Gear-1 Driving School, aiming to provide an interactive and engaging platform for learning traffic rules and proper driving behavior. Players navigate through simulations and quizzes that reinforce theoretical driving concepts through hands-

on gameplay. CodeBound shifts to a 2D format to prioritize accessibility and performance on lower-end devices, while its puzzles are grounded in established computer science curricula, ensuring educational credibility through consultation with academic resources and expert feedback. Its storytelling approach, where players navigate a mysterious digital tower, makes learning programming engaging and distinct from Lakbay's 3D driving focus.

According to Batoon et al.[16] entitled "Mathematics Integration on the Development of a 3D Action Role-Playing Game: The Light of End," the researchers developed an educational action RPG integrating simple addition problems to enhance players' mental computation skills during gameplay. Built using Unreal Engine 4 and Blender 2.8, the game incorporates Philippine mythological characters and emphasizes real-time combat strategy. Players solve math problems to regenerate stamina during battles, combining entertainment with educational value. The system, developed using RAD methodology, received an overall evaluation rating of "Excellent" with a mean score of 4.76 from IT professionals, students, and general users

To enhance this system, the study adopts a 2D environment to deliver a broader range of programming concepts, from variables to functions, through a curriculum-synced, story-driven adventure. Players solve puzzles to ascend 100 floors, with each level reinforcing academic learning objectives. This narrative depth and curriculum alignment provide a more comprehensive educational experience compared to The Light of End's arithmetic focus.

Foreign Literature/Studies

Cheung and Ng[17] In this study highlights the positive impact of the use of the game PaGamO on learning. The game helped the students perform better on exams. While students generally found the game engaging and fun, especially for mobile learning, its appeal was not universal, with some preferring traditional study methods. However, they noted that while many students enjoyed the interactive and mobile-friendly approach, some still preferred traditional study methods.

To resolve this weakness, CodeBound goes overcomes this by requiring players to write actual code in a 2D puzzle-based adventure, aligned with computer science curricula. Its storytelling immerses players in a digital tower, solving coding challenges in Python, Java, or C#. This hands-on, narrative-driven approach fosters deeper understanding compared to PaGamO's question-based format.

Tacouri and Nagowah[18] developed the game in response to challenges posed by online learning during the COVID-19 pandemic. The game aims to make self-learning more effective by integrating educational content with gameplay. Code Saga is a serious mobile game designed to help undergraduate IT students learn programming in an engaging and interactive way. Their study demonstrates the effectiveness of integrating programming concepts into a serious mobile game to make learning more engaging for IT students.

The study addresses this by offering 100 levels of increasing complexity in a 2D environment, structured around a curriculum-synced narrative. Players progress through a digital tower, solving puzzles that evolve from basic variables to advanced functions. This

structured progression and storytelling enhance engagement and learning depth compared to Code Saga.

Vahldick et al. [18] developed NoBug's SnackBar a serious game designed to help students develop problem-solving skills in introductory programming courses. It was developed using an iterative and interactive methodology, allowing continuous improvements based on feedback from multiple testing phases across different semesters. The game provides a flexible learning experience, enabling students to engage with it at their own pace without time or location constraints. This approach aims to create a more realistic assessment of students' learning progress and engagement.

To address these weaknesses, CodeBound introduces programming concepts gradually across 100 levels in a 2D adventure, with each floor focusing on a single concept like loops or conditionals. Its curriculum alignment ensures relevance, and the storytelling narrative keeps players engaged, preventing the overwhelm noted in NoBug's SnackBar.

Byusa et al. [19] conducted a systematic review titled Game-based learning approach on students' motivation and understanding of chemistry concepts, examining 57 studies published between 2010 and 2021. The review focused on 24 educational games used primarily at the secondary and university levels, categorized into digital, non-digital, and hybrid formats. The findings revealed that game-based learning (GBL) enhances students' motivation, participation, and conceptual understanding in chemistry.

CodeBound mitigates this by balancing fun with education through a curriculum-synced, 2D puzzle-based adventure. Its narrative-driven structure ensures players engage deeply with programming concepts, reinforced by in-game feedback and hints, making it more pedagogically robust than chemistry-focused games.

Demiray et al. [20] developed Flood Game, a web-based 3D serious game designed to raise awareness and enhance understanding of flood risk mitigation. The game enables users to simulate real-world flood scenarios, apply various mitigation strategies, and observe their impacts on communities through an interactive and immersive environment. The study concluded that FloodGame effectively promotes disaster awareness and decision-making skills by providing a realistic, engaging platform to explore and evaluate flood mitigation measures.

To address the weaknesses the study uses a 2D environment to ensure offline accessibility on low-end devices, with puzzles aligned to computer science curricula. Its storytelling, where players escape a digital tower through coding, enhances engagement without internet dependency, making it more accessible than Flood Game.

2.2 Conceptual Framework

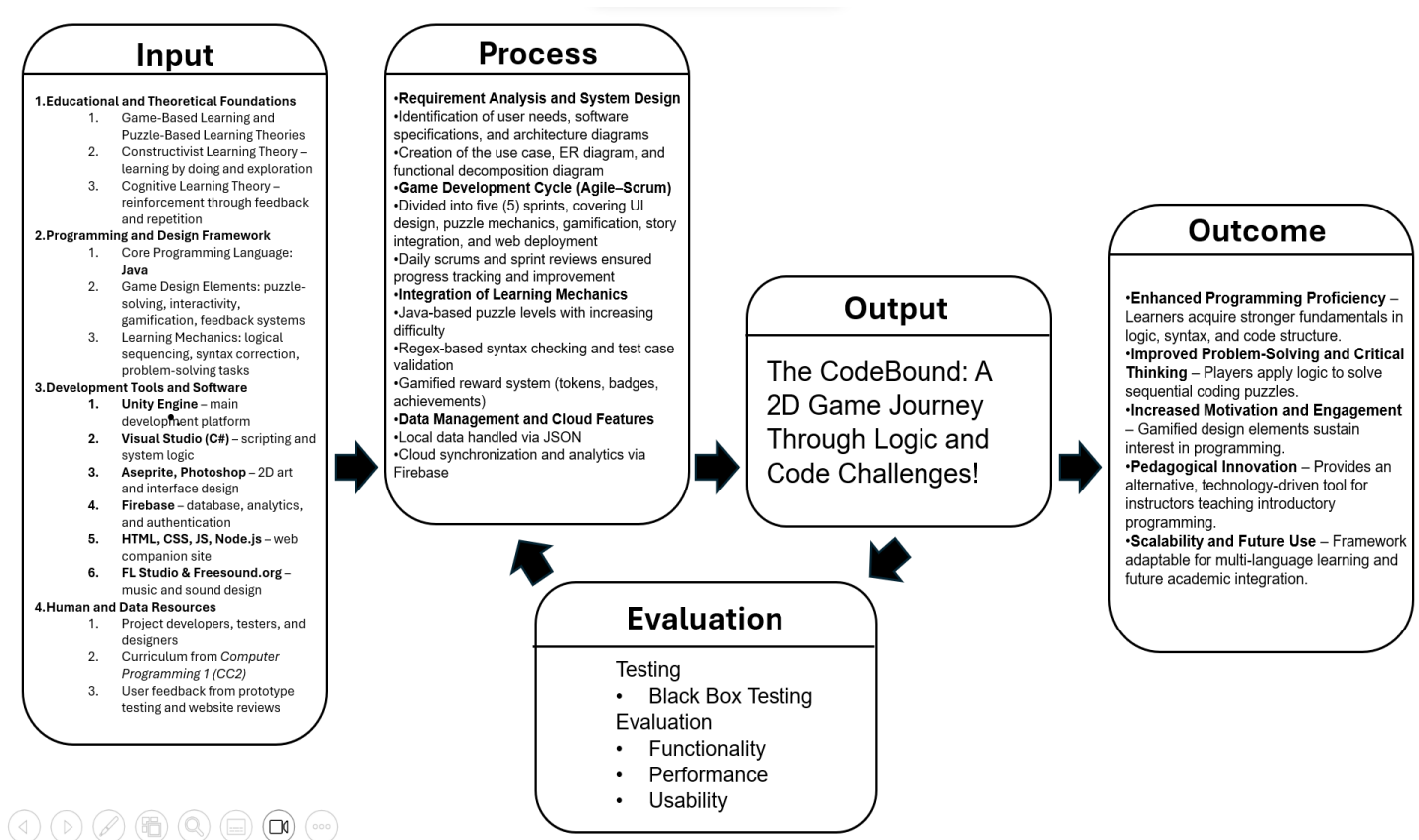


Figure 1. Conceptual Framework of CodeBound

The conceptual framework of CodeBound: A 2D Game Journey Through Logic and Code Challenges! illustrated the overall flow of the study, showing how the project's resources, processes, and outcomes were systematically connected. The framework followed the Input–Process–Output–Evaluation–Outcome (IPOEO) model, which demonstrated the development cycle and educational impact of the system.

The Input stage consisted of the essential elements required to develop the project, including educational and theoretical foundations such as game-based and constructivist learning theories, the programming and design framework using Java and gamification principles, and the development tools and software utilized in creating both the game and the web platform. It also involved human and data resources, which included the project team, programming curriculum, and feedback gathered from prototype testing.

The Process stage represented the development methodology and the core

activities undertaken in producing the system. Guided by the Agile–Scrum approach, this phase involved requirement analysis, system design, iterative game development, and the integration of learning mechanics such as syntax validation and puzzle-based progression. Cloud integration through Firebase ensured data storage, analytics, and real-time synchronization throughout the development.

The Output phase presented the tangible results of the process—specifically, the development of CodeBound, a 2D educational puzzle game designed to teach Java programming through interactive challenges, along with its companion website and administrative dashboard that supported user engagement and performance monitoring.

The Evaluation phase assessed the functionality, usability, and overall performance of the system. It included black box testing, performance assessments, and user evaluations focusing on functionality, usability, and educational effectiveness. Feedback gathered from students and instructors served as the foundation for refinement and improvement.

Finally, the Outcome stage highlighted the educational and cognitive benefits gained through the system’s implementation. These outcomes included enhanced programming proficiency, improved problem-solving and logical thinking, increased motivation and engagement through gamification, and pedagogical innovation that supported modern teaching strategies. The framework also emphasized scalability and future use, ensuring adaptability for multi-language programming instruction and future academic integration

3 TECHNICAL BACKGROUND

3.1 Software Development Requirements

The software selected was directly aligned with the development of both the CodeBound: A 2D Game Journey Through Logic and Code Challenges! educational game and its supporting web application. Each software component contributed to the realization of the project's core features, such as multi-language programming support, immersive 2D environments, in-game logic validation, and a responsive, interactive landing page.

Table 3. Software Development Requirements

Software	Purpose in Development
Unity	Core game engine used to develop the 2D gameplay system, puzzle logic, terminal interactions, movement, and the audiovisual environment. Unity's UI Toolkit (UXML & USS) was used to design and manage all game and in-game interfaces (menus, profile, achievements, shop) using HTML/CSS-like styling.
Visual Studio (C#)	Functioned as the main IDE for scripting game logic, including code parsing, puzzle validation, NPC behaviors, inventory systems, and level progression.
Aseprite	Used for creating and animating 2D character sprites (players and NPCs), designing interactive environment elements, terminal objects, and other in-game pixel art assets.

Adobe Photoshop	Utilized for designing GUI elements such as badges, tokens, icons, loading screens, logos, and character outfit textures.
FL Studio 21	Employed for composing original background music and soundtracks to enhance immersion, alongside creating complementary sound effects (e.g., button clicks).
Freesound.org	Provided additional royalty-free sound effects (e.g., terminal feedback, footsteps, level completion sounds).
HTML, CSS, JavaScript	Used in designing the web companion site, which included overview pages, a download section, FAQ, and community hub.
PHP / Node.js	Implemented for general backend routing and dynamic web functionalities of the companion site, supporting user registration and authentication flow.
UXML	Defined the structure and layout of UI elements within Unity's UI Toolkit system, such as buttons, menus, and panels.
Firebase	Implemented as the core cloud database solution for storing and synchronizing critical persistent data (user profiles, achievements, tokens) and enabling Admin Console functionalities (authentication, dynamic content, analytics, and reporting).

The development of the game utilized a range of software tools. Unity served as the primary game engine, responsible for building the 2D environment, managing assets, animations, and implementing puzzle interactions. For the game's user interface, the Unity UI Toolkit was employed, which enabled the creation of structured UI layouts with UXML and styled components using USS. This allowed for a consistent and cohesive appearance across menus, panels, and the in-game terminal. All gameplay logic, player controls, and puzzle mechanics were programmed in Java, written and managed within Visual Studio, which functioned as the project's main integrated development environment (IDE).

Visual assets such as UI backgrounds, icons, and logos were created and edited in Adobe Photoshop, while Aseprite was used for 2D modeling and animating game characters and environmental props. These assets were then imported into Unity to be integrated into the game. To enhance the auditory experience, sound effects were sourced from Freesound.org, and original background music and soundtracks were composed and produced in FL Studio 21.

The CodeBound website was developed using HTML, which defined the overall structure and layout of the different pages, including the navigation bar, Home, Trailer, FAQ, Reviews, and Community sections. HTML provided the foundational elements that organized the content and ensured readability. CSS was implemented to design the visual appearance of the website by controlling colors, fonts, layouts, and positioning of elements such as the background image, character image, and footer. It was also used to ensure responsiveness, allowing the site to adjust seamlessly across various screen sizes such as phones, tablets, laptops, and desktops. JavaScript was incorporated to add interactivity and functionality to the site. Furthermore, Node.js was used to handle the database operations for the login/signup system and the leaderboard, which tracked and displayed user progress in the game.

3.2Hardware Development

The defined hardware specifications were intentionally aligned with the minimum requirements necessary to run CodeBound: A 2D Game Journey Through Logic and Code Challenges! effectively across both desktop and mobile platforms. These baseline configurations ensured that the game delivered its core features—such as immersive 2D environments, interactive puzzles, and a responsive user interface—without compromising overall performance or stability.

Table 4. Hardware Development Requirements

Hardware	Specification
Desktop/Laptop	Processor: I3 / Ryzen 3 Gpu: Integrated Vega 8 / Nvidia Gt 1030 Memory: 8gb Ram Ssd: 256gb Sata / M.2 Motherboard: A320 Psu: 400w+ Os: Windows 10
Mobile	Processor: Snapdragon 665 / Mediatek Helio G80 Ram: 4gb Storage: At Least 8gb Free Android: Android 9

3.3 Peopleware

The project requires a multidisciplinary team with defined roles and responsibilities to ensure a successful game and website deployment.

Table 5. Peopleware

Role	Team Member(s)	Responsibilities
Project Manager	Elaika Joy S. Santiago	Oversees project timeline, documentation, coordination among team.
Lead Developer	John Lester P. Tiglao	Handled Unity scripting, logic evaluation, puzzle systems, player mechanics.
3D Artist / Animator	John Lester P. Tiglao, Mark Guiller Melchor	Designed 2D environments, characters, animations using Aseprite.
UI/UX Designer	All members	Created game interface and website layout; ensures responsive design and usability.
Web Developer	Mark Guiller Melchor	Built the landing page using HTML, CSS, JS. Implements download links, feedback form.
Game Tester / QA	All members	Conducted gameplay testing, bug tracking, and performance evaluation.
Audio Designer	Elaika Joy S. Santiago	Produced customized background music and SFX via FL Studio.

Documentation Specialist	Elaika Joy S. Santiago,	Managed writted documentation, thesis formatting, and test logs.
--------------------------	-------------------------	--

3.4 Source of Data

The data for this project were sourced primarily through internet-based gathering methods, which included the collection of publicly available datasets, documentation, and coding examples related to the supported programming language, Java. These materials were utilized to design, populate, and validate the structure and content of the in-game programming challenges.

A major academic data source was the subject Computer Programming 1 (CC2). The course topics, lectures, and instructional materials served as the foundation for designing puzzle challenges and programming lessons integrated into the game. Aligning the game content with this course ensured that the educational material adhered to established academic standards and supported the learning objectives outlined in the curriculum.

The game itself functioned as both a product and a source of practical data. Developed as a single-player, 2D puzzle-based learning experience using the Unity game engine, it generated structured gameplay interactions that were observed and tested during development. Unity provided the core tools for creating interactive 2D environments, while Aseprite was employed to design and model 2D assets such as characters, obstacles, and puzzle components.

Player data were tracked and stored locally using JSON (JavaScript Object

Notation), a lightweight and human-readable format. This enabled the system to record essential information such as completed levels and overall progression. The use of JSON ensured that players could seamlessly resume gameplay while maintaining an efficient and reliable data management process within the game.

In addition to in-game data, external user feedback was collected through the project's companion website. Players could submit star ratings and written reviews, which were periodically monitored to assess user satisfaction, identify usability issues, and guide future updates. Although these external reviews were not directly integrated into the in-game data system, they provided valuable insights that informed evaluation and improvement efforts in later development stage

4 METHODOLOGY

This chapter explains the methods used to design and develop CodeBound: A 2D Game Journey Through Logic and Code Challenges, a game created to help students improve their programming skills. It describes how the researchers gathered data, designed and developed the system, and evaluated its effectiveness. The team used a structured process to collect information, build the game, and assess its results. Each method was selected based on the study's goals, the system's requirements, and the needs of its users. The chapter also highlights how data and expert insights guided the development of CodeBound to make it engaging, educational, and effective for learners.

Methods in Data Gathering

4.1.1 Internet Method

The internet was the main source of information for developing CodeBound: A 2D Game Journey Through Logic and Code Challenges. The researchers used credible online sources such as academic journals, digital libraries, developer forums, and tutorials to gather insights on game design, puzzle-based learning, and 2D interactive environments. Studies from Google Scholar and IEEE Xplore, as well as examples from CodeCombat, CodinGame, and Minecraft: Education Edition, helped guide the game's structure and learning approach. Local projects like Anito: Battle of the Gods, AGILA, and PortaLogica also provided inspiration. Developer forums like Unity Forum, Stack Overflow, and Reddit's r/gamedev, along with tutorials from YouTube and Udemy, supported team solving technical issues and improved their skills. The collected online information shaped the game's design, mechanics, and learning features, ensuring it effectively develops students logical thinking and problem-solving skills.

4.1.2 Observation

The observation method was used to study existing puzzle-based games and their effects on user engagement and problem-solving skills. The researchers conducted observations at Tarlac State University, examining different educational and puzzle games to identify effective design elements and gameplay mechanics. The goal was to see how players interacted with coding puzzles, how difficulty levels affected learning, and how 2D environments supported engagement and spatial awareness. The researchers focused on game interfaces, user interactions, feedback, and learning progression. Through these observations, the team identified key features of effective educational games such as simple controls, clear instructions, adaptive difficulty, and rewarding progress. The findings guided the design of CodeBound, helping ensure it applied proven game-based learning techniques while avoiding common usability issues.

4.2 To design develop CodeBound and its Web Application featuring core gameplay(puzzles, code-based interactions, levels), gamification elements (tokens, achievements, characters skins), and community components (download link, hub, FAQs, trailer).

4.2.1 Requirement Analysis and Documentation

4.2.1.1 User Requirements

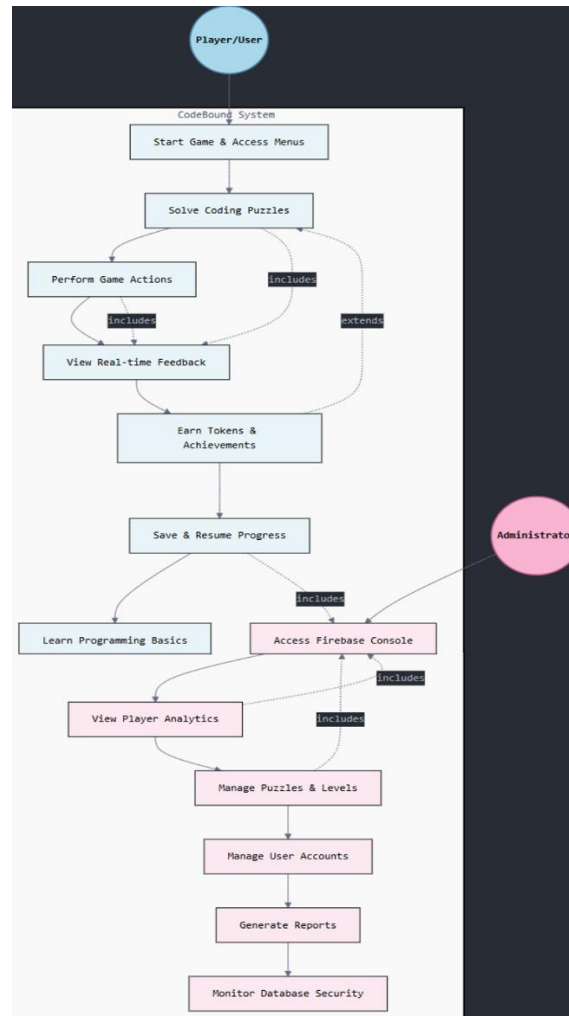


Figure 2. Use Case Diagram of CodeBound

The Use Case Diagram illustrated the interactions between the users and the system's functionalities within the 2D puzzle-based coding game. It provided a visual representation of the various actions that the player, administrator, and system performed, offering a clearer understanding of the overall system behavior and user relationships.

4.2.1.2 User Characteristics

The CodeBound: A 2D Game Journey Through Logic and Code Challenges! has two main types of users: Player (User) and Administrator (Admin), each with specific roles and system access.

Player(User):

The player interacts directly with the game for learning and entertainment. They can:

1. Start the game and access menus like settings, progress, and saved profiles.
2. Solve coding puzzles using the in-game terminal and logic mechanics.
3. Perform actions such as moving characters, triggering switches, and completing objectives.
4. View real-time feedback and syntax validation.
5. Earn tokens and achievements and use them for customization.
6. Save and resume game progress locally, with key data synced to Firebase.
7. Learn programming basics such as syntax, loops, and conditionals through puzzles.

This module supports self-paced learning, allowing players to improve programming and problem-solving skills while progressing through levels.

Administrator(Admin):

The admin manages the system through Firebase. They can:

1. Access the Firebase Console for monitoring and data management.
2. View player analytics, including level completions and achievements.
3. Add, update, or delete puzzles and levels.
4. Manage user accounts and maintain database accuracy.
5. Generate reports and monitor educational outcomes.

6. Ensure secure and validated database operations.

The admin module ensures that data is synchronized, secure, and used to track learning progress and improve the game's overall performance supporting the project's goal of delivering an effective, engaging, and educational 2D puzzle-based game.

4.2.1.3 Functional Requirements

This section presents the functional requirements of CodeBound: A 2D Game Journey Through Logic and Code Challenges, an educational puzzle game that teaches programming through interactive code-solving tasks. These requirements define the main features, actions, and behaviors expected from users. Each requirement is categorized by **priority** (High, Medium, Low) and **complexity** (Low, Medium, High), based on the project objectives and design mockups. They serve as a guide to ensure that all developed features support a smooth, engaging, and learner-focused experience.

Table 6. Functional Requirement for User Module

Req.ID	Requirement Description	Priority	Complexity
FR1.	The system shall allow players to launch the game and access the main menu.	High	Low
FR2.	Players shall be able to create and select save files to store progress and settings.	High	Medium
FR3.	Players shall solve coding puzzles by writing and submitting code through the in-game terminal.	High	Medium
FR4.	The system shall evaluate submitted code with syntax checking and test validation, then provide feedback.	Medium	Medium
FR5.	Players shall view real-time feedback for syntax errors and logic output.	Medium	Low
FR6.	Players shall access hints during gameplay to assist with puzzles.	Medium	Low

FR7.	The system shall track player progress and display completed, unlocked and locked levels.	High	Medium
FR8.	Players shall retry failed levels without losing overall progress.	Medium	Low
FR9.	The system shall award tokens and achievements based on performance and sync them with the cloud database.	High	Medium
FR10.	Players shall use tokens in the in-game shop to unlock cosmetics upgrades and customizations.	Medium	Medium
FR11.	The game shall unlock levels sequentially to guide players through a structured learning path.	High	Medium
FR12.	The system shall save and load player data locally using JSON, while syncing tokens and achievements with Firebase.	High	Medium
FR13.	The system shall allow players to log in or register via the CodeBound web application to access game downloads, community hub, and profile data.	High	Medium

Table 7. Functional Requirement for Admin Module

Req.ID	Requirement Description	Priority	Complexity
FR14.	The system shall allow the admin to log in securely through Firebase Authentication.	High	Medium
FR15.	The admin shall view player activity and analytics stored in Firebase.	High	Medium
FR16.	The admin shall add, edit, or delete level and puzzles through the Firebase Realtime Database.	Medium	Medium
FR17.	The system shall allow the admin to manage user records including registration and removal.	Medium	High
FR18.	The admin shall generate and export reports on player performance data from Firebase.	Medium	Medium
FR19.	The system shall ensure all Firebase operations are secured and verified through authentication.	High	Medium

4.2.1.4 Non-Functional Requirements

This section defines the non-functional requirements of CodeBound, focusing on performance, security, usability, and compatibility. These requirements ensure the system runs smoothly, protects user data, and delivers a consistent learning experience across devices.

**Table 8. Non-Functional Requirement for Safety and Security of The
CodeBound: A 2D Game Journey Through Logic and Code
Challenges!**

Code	Dependencies Description
NFR1	Player layer progress, including completed levels and settings, must be saved locally and synced with the Firebase cloud database.
NFR2	Local data such as level status and preferences must be securely stored, while cloud data (achievements, profiles) is protected through Firebase authentication and security rules.
NFR3	In-game activities like level completion, hint use, and token spending must be logged for debugging and feedback purposes.
NFR4	Player data (profiles, achievements, tokens) must be automatically backed up and restored through Firebase.
NFR5	The game must maintain a minimum of 30 frames per second (FPS) to ensure smooth gameplay.
NFR6	The system must provide an intuitive interface with clear instructions, readable text, and easy-to-understand hints for all user levels.
NFR7	Main screens such as the menu, level interface, and terminal must load within 3 seconds to improve user experience.
NFR8	The game must be compatible with both desktop and mobile platforms, ensuring consistent performance and display.
NFR9	The Firebase Realtime Database must store and retrieve administrative data reliably with minimal delay.
NFR10	Admin access to Firebase must be protected using secure email-password authentication.
NFR11	The Firebase database must update in real-time whenever changes are made by the admin.
NFR12	Database access must be restricted to authorized admin accounts only.

4.2.2 Design of Software and/or System and/or Product and/or Processes

4.2.2.1 ER Diagram

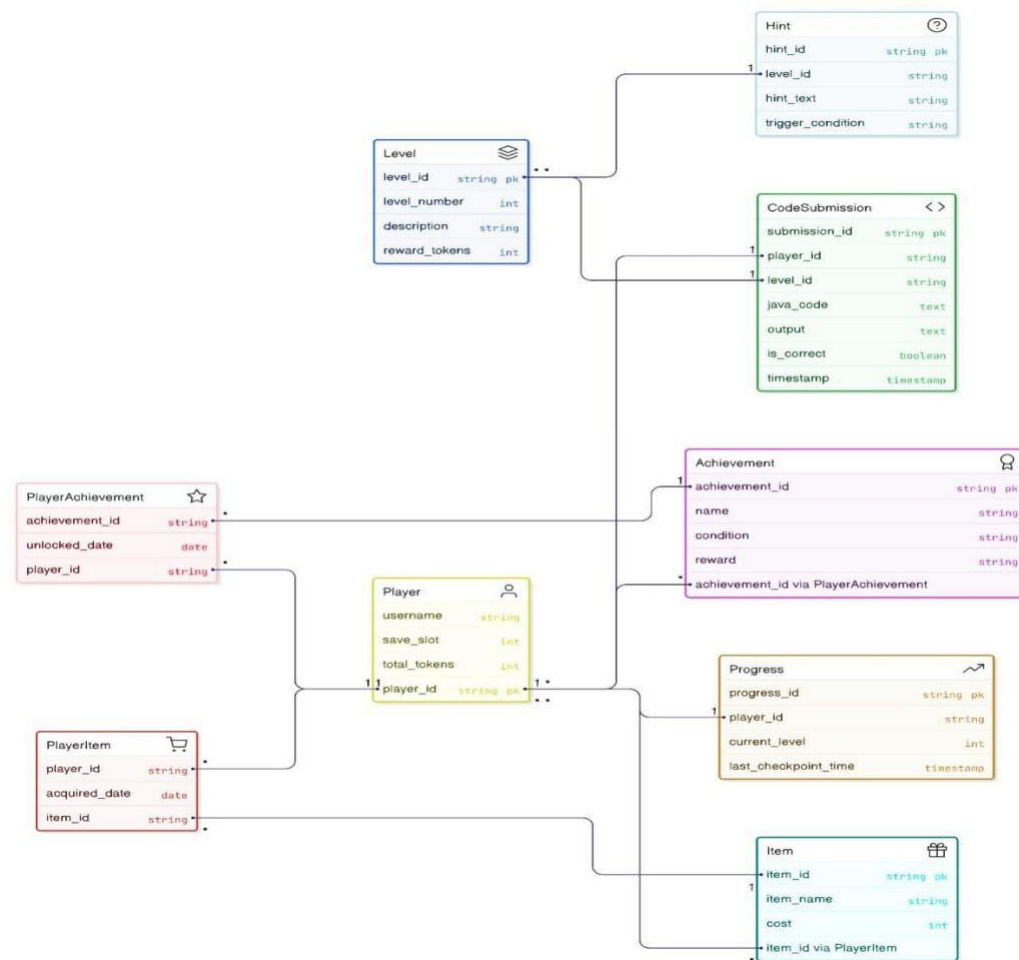


Figure 3. ERD Diagram of Codebound

4.2.2.2 Functional Decomposition Diagram

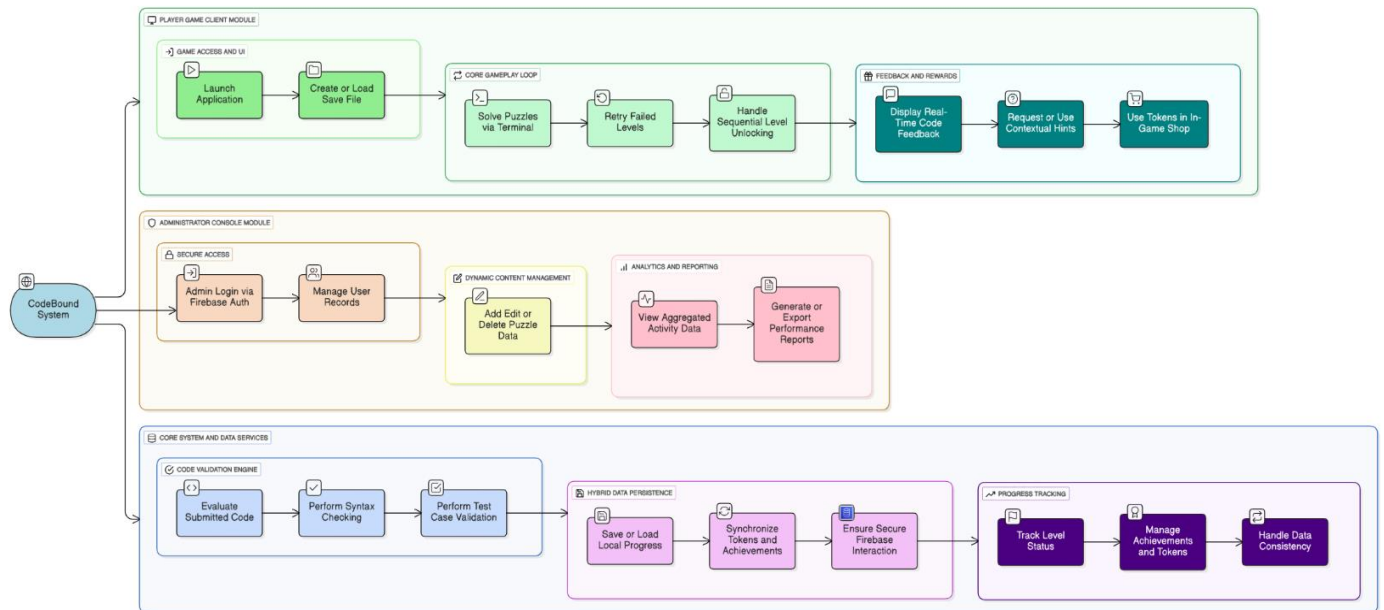


Figure 4. Functional Decomposition Diagram of Codebound

The functional decomposition diagram shows how CodeBound is divided into major components: Core Gameplay, User Management, Gamification and Feedback, and Admin Controls. The Core Gameplay manages puzzles, code validation, and level progressions. User Management handles profiles, saving/loading and Firebase data synchronization. Gamification and Feedback track rewards and progress. Admin Control allows secure access to manage content and user data through Firebase.

4.2.2.3 Data Storage Architecture

The data storage of CodeBound: A 2D Game Journey Through Logic and Code Challenges! utilized Firebase, a cloud-based database system provided by Google. Firebase was chosen for its scalability, ease of integration with Unity, and capability to support real-time data synchronization between the game and its companion web application.

The system used Firebase Realtime Database to store and retrieve persistent player data, specifically user profiles, achievements, and tokens earned. While immediate in-game session progress is stored locally, this cloud-based architecture enabled the game to update high-level user information dynamically, ensuring critical milestones are backed up and synchronized without requiring manual database management.

In addition, Firebase Authentication was implemented to handle secure user login and registration, ensuring that each player's progress was uniquely tracked and protected.

The integration also supported future scalability, allowing multiple users to register and store their data simultaneously without compromising system performance.

The Firebase database architecture consisted of three primary components:

1. User Data Node – stored unique player IDs, usernames, and authentication credentials.
2. Progress Node – recorded level completions, scores, and achievements for each player.
3. Leaderboard Node – maintained ranking data for competitive performance and community engagement.

By integrating Firebase, CodeBound achieved efficient, reliable, and secure data management. This ensured that users could seamlessly access their progress and achievements across devices while maintaining data consistency and security.

4.2.2.4 Operating Environment

This section lists the required tools, platforms, and specifications for developing and running CodeBound. The game is built in Unity using C#, with assets designed in Aseprite and data managed through Firebase. It supports both Windows and Android devices for smooth gameplay and accessibility.

Table 9. Operating Environment

Code	Environment Description
OE1	The game shall run on Windows desktop and laptop devices.
OE2	A mobile version shall be available for Android smartphones and tablets.
OE3	Unity serves as the main game engine for building and deploying the 2D environment.
OE4	Aseprite is used to design 2D assets, characters, and environments.
OE5	C# is the core scripting language used within Unity.
OE6	The system requires at least 4GB of RAM for stable performance.
OE7	The game shall support local storage for saving players' progress and settings.
OE8	Admin functions require an internet connection to access Firebase for data management.
OE9	Internet is only needed for downloading or admin monitoring; gameplay remains offline supported.

4.2.2.5 Design and Implementation Constraints

This section outlined the key design and implementation constraints that influenced the development of CodeBound: A 2D Game Journey Through Logic and Code Challenges. These constraints were shaped by technical limitations, platform dependencies, tool capabilities, and the development team's expertise. Recognizing these limitations was essential in setting realistic expectations, ensuring focused development, and making informed design decisions throughout the project life cycle. The game was developed using the Unity engine and deployed exclusively on Windows and Android platforms. Due to time, experience, and resource limitations, certain features—such as multiplayer functionality, web-based compatibility, and dynamic asset generation—were intentionally excluded to maintain the project's scope and ensure overall quality, stability, and performance within achievable parameters.

Table 10. Design and Implementation Constraints

Code	Design Constrains and Implementation Constraints Description
DC1	The game is supported only on Windows PCs and Android devices; iOS and browser-based versions are not included.
DC2	Unity is used as the main development engine; no alternative engines or frameworks are used.
DC3	Visual assets and sprites are created using Aseprite and Unity tools, with limited complexity due to time and resource constraints.
DC4	Firebase is used only for admin functions such as data monitoring and analytics; players cannot access the database directly
IC1	The game is single-player only; multiplayer and online connectivity are not included.
IC2	Player progress is stored locally, while tokens and achievements are synchronized with Firebase for consistency.
IC3	The total game size is limited to under 1 GB to support smooth installation and mobile accessibility.
IC4	An internet connection is required only for admin access to Firebase; gameplay remains fully offline.

4.2.3 System Development Methodology

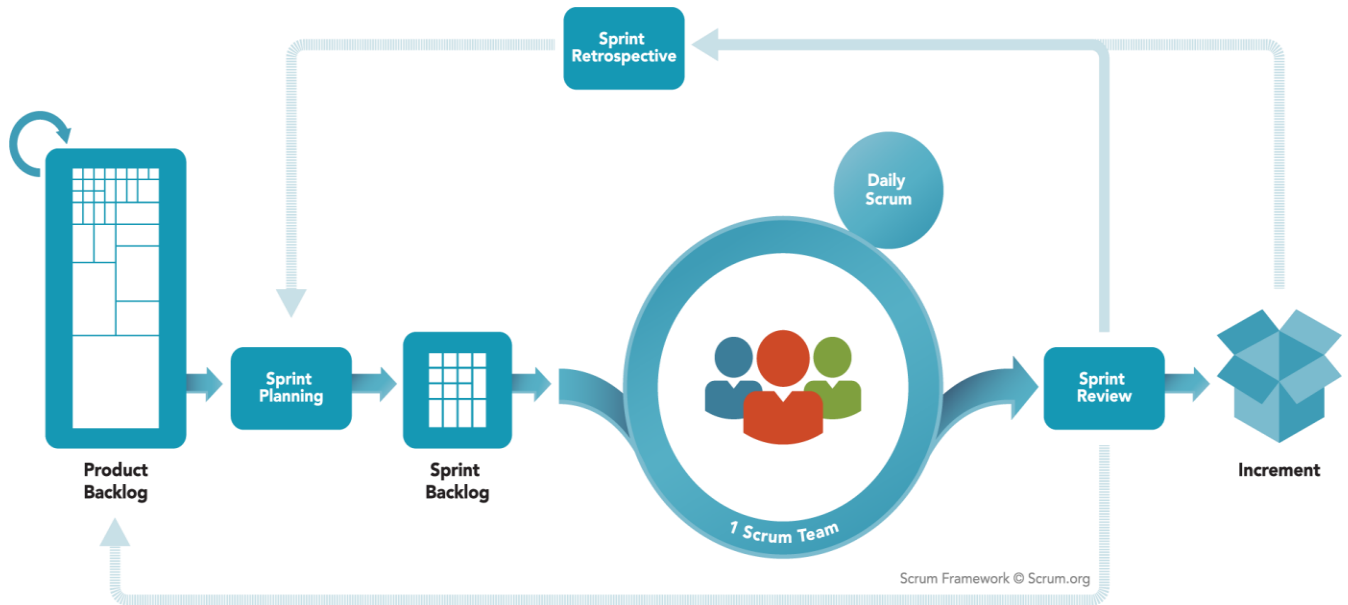


Figure 5. Agile Scrum Methodology

The figure shows the Agile-Scrum Framework used in developing **CodeBound: A 2D Game Journey Through Logic and Code Challenges**. This method focused on teamwork, iteration, and continuous improvement. The process started with the **Product Backlog**, which listed all the needed features and tasks. During **Sprint Planning**, selected items were moved to the **Sprint Backlog** to set clear goals for each sprint. The team worked together daily, holding **Daily Scrums** to track progress and solve issues. After each sprint, a **Sprint Review** was done to present the output and gather feedback, followed by a **Sprint Retrospective** to reflect and improve. Each sprint produced a working part of the game ready for integration. This process ensured the game was developed step-by-step to meet the project's main goal of creating a functional and engaging 2D educational game.

Product Backlog

The Product Backlog served as the main reference for developing CodeBound: A 2D Game Journey Through Logic and Code Challenges. It listed all essential features, tasks, and improvements needed to complete the game. Each backlog item represented a specific function or activity such as interface design, game mechanics, level creation, reward system, testing, and deployment. These items were organized by priority High, Medium, or Low to help the team focus on the most important goals first. Estimated durations for each task, measured in days, were included to guide sprint planning and ensure efficient workflow management. The backlog was continuously updated throughout the development process as feedback was received and tasks were completed. This flexibility allowed the team to improve the system, adjust priorities, and ensure steady progress in line with the project's objectives.

Table 11. Product Backlog

Sprints	Product Backlog	Priority	Duration
1	Design main menu and save file UI	High	5 days
1	Create terminal interface and level screen	High	5 days
1	Design 2D character sprites and environment tiles	High	5 days
2	Implement terminal input and code parsing system	High	7 days
2	Develop puzzle logic and syntax validation	High	4 days
3	Create and balance levels 1–20	High	8 days
3	Integrate token reward system	Medium	6 days
3	Implement achievements and badge system	Medium	7 days
4	Add in-game feedback and hint system	High	4 days
3	Set up player progress tracking and local save feature	Medium	5 days
3	Integrate Firebase for Admin Analytics and Data Storage	High	5 days
3	Configure Firebase Authentication for Secure Login (Admin Only)	High	5 days
3	Connect Game Data with Firebase for Cloud Sync and Leaderboards	High	5 days

4	Create narrative cutscenes and game intro	Low	7 days
4	Conduct internal playtesting and gather user feedback	High	5 days
4	Perform bug fixing and gameplay polish	High	5 days
5	Develop game website and download page	Medium	5 days
5	Write and design FAQ, About, and Review pages	Low	5 days
5	Build and export PC installer and Android APK	High	7 days
5	Final performance optimization and release polish	High	6 days

Sprint Planning

Sprint Planning was done before each sprint to choose specific tasks from the product backlog that could be completed within the set time. The team defined the sprint goal, assigned tasks based on skill, and divided larger features into smaller, manageable parts. During this stage, the team discussed task duration, technical needs, and acceptance criteria to ensure everyone understood the plan. They also considered factors like academic schedules, adviser feedback, and possible risks. This process helped the team stay organized, focused, and realistic in achieving their goals for each sprint.

Sprint Backlog

The Sprint Backlog contained the specific tasks chosen during Sprint Planning. It acted as the team's guide for what to develop, test, and integrate within each sprint. For CodeBound, this included creating levels, coding terminal logic, and improving the user interface. The backlog was regularly updated as the team made progress. Any new ideas, changes, or challenges were noted and used to plan future sprints more effectively.

Daily Scrum

The Daily Scrum was a short 10–15 minute meeting held each day of the sprint. Team members shared what they finished yesterday, what they planned to do today, and any problems they faced. These quick updates kept everyone on the same page, helped solve issues early, and ensured steady progress toward the sprint goals..

Sprint Review

The Sprint Review allowed the team to present finished work and gather feedback from instructors, peers, and testers. The team demonstrated new features such as terminal mechanics, levels, and interface updates through live demos or recorded videos. Feedback on usability and design was collected, and any suggested changes were added back to the Product Backlog. This process helped ensure the game stayed aligned with learning goals and user needs.

Sprint Retrospective

After each Sprint Review, the team held a Sprint Retrospective to reflect on what went well, what didn't, and how to improve. They discussed communication, task management, and workflow efficiency. For example, the team improved task tracking in Trello and refined playtesting methods for better feedback. Insights from each retrospective were applied to the next sprint to boost teamwork and development quality.

Increment

Each sprint produced a usable version of the game with newly developed features such as UI updates, new levels, or gameplay improvements. The final sprint combined all components including Firebase integration, user analytics, and performance optimization into a complete and fully playable version of CodeBound. This iterative process kept the game stable, testable, and aligned with its educational and functional goals.

4.2.3.1 Planning

Sprint Planning marked the start of each development cycle for CodeBound: A 2D Game Journey Through Logic and Code Challenges. In this phase, the team reviewed the product backlog and selected specific tasks that could be completed within the sprint's timeframe. The goal was to ensure each sprint contributed directly to the game's design, functionality, and learning objectives. Tasks were carefully prioritized based on importance and feasibility. The team divided larger features into smaller, manageable tasks and assigned them according to individual skills such as game logic programming, interface design, asset creation, and database setup. Estimated durations and timelines were also defined to maintain a clear and achievable workflow.

Each sprint focused on producing a functional version of the game that integrated new features or improvements. These increments were tested, refined, and reviewed to ensure quality, usability, and alignment with CodeBound's educational purpose. Tables 12 to 16 present the sprint schedules, task assignments, and development timelines from August to November 2025.

Table 12. Sprint 1: UI and 3D Asset Design

Task	Responsible	Duration
Design Main Menu, Save File UI	Tiglao, John Lester P.	Aug 1–5
Create terminal interface and level screen	Tiglao, John Lester P.	Aug 6–10
Design 2D character sprites and environment tiles	Tiglao, John Lester P.	Aug 11–14
Create Level Objects & Environments	Melchor, Mark Guilller	Aug 11–15
Design Achievement/Shop UI	Mabanglo, Eowyn Feona F.	Aug 11–16
Final Adjustments	Entire Team	Aug 17

Table 12 outlined Sprint 1, which focused on the design of the user interface and 2D game assets. The tasks included the creation of the main menu, terminal screen, character models, environmental tiles, and the shop interface. Final user interface adjustments were completed by the team before proceeding to the next sprint.

Table 13. Sprint 2: Game Environment & Puzzle Logic

Task	Responsible	Duration
Unity Environment & Lighting Setup	Tiglao, John Lester P.	Aug 18–21
Implement terminal input and code parsing system	Tiglao, John Lester P.	Aug 22–28
Player controls and movement testing	Tiglao, John Lester P.	Aug 29 – Sep 3
Develop puzzle logic and syntax validation	Tiglao, John Lester P.	Sep 4–7

Table 13 presented Sprint 2, which centered on the implementation of the game environment, logic components, and terminal interaction. The environment was constructed in Unity, followed by the development of puzzle validation logic and user controls.

Table 14. Sprint 3: Level Expansion & Gamification

Task	Responsible	Duration
Create and balance levels 1–20	Tiglao, John Lester P.	Sep 8–15
Integrate token reward system	Melchor, Mark Guiller	Sep 8–13
Implement achievements and badge system	Mabanglo, Eowyn Feona F.	Sep 14–20
Progress tracking and game data saving	Santiago, Elaika Joy S.	Sep 21–28
Integrate Firebase for Admin Analytics and Cloud Data Management	Tiglao, John Lester P.	Sep 24-28

Table 14 showed Sprint 3, which expanded the gameplay experience through the addition of levels 6 to 20 and the integration of gamification features such as tokens, achievements, and progress tracking. These components enhanced player motivation and supported greater learning retention.

Table 15. Sprint 4: Story Integration, Feedback, Testing

Task	Responsible	Duration
Create narrative cutscenes and game intro	Mabanglo, Eowyn Feona F.	Sep 29 – Oct 5
Add in-game feedback and hint system	Santiago, Elaika Joy S.	Oct 6–9
Conduct internal playtesting and gather feedback	Entire Team	Oct 10–14
Perform bug fixing and gameplay polish	Entire Team	Oct 15–19

Sprint 4, detailed in Table 15, focused on narrative integration, user feedback systems, and internal testing. The cutscenes and story elements were developed, and a feedback mechanism was added to enhance in-game learning. During this phase, the team conducted user playtesting and performed bug fixing to ensure game stability and quality.

Table 16. Sprint 5: Website, Installer & Final Polish

Task	Responsible	Duration
Develop game website and download page	Santiago, Elaika Joy S. Melchor, Mark Guiller	Oct 20–24
Write and design FAQ, About, Leaderboard and Review pages	Mabanglo, Eowyn Feona F.	Oct 25–29
Build and export PC installer and Android APK	Tiglao, John Lester P.	Oct 25 – Nov 2
Finalize Firebase Configuration for Admin Dashboard and Analytics Reports	Tiglao, John Lester P.	Nov 1-5
Final performance optimization and release polish	Entire Team	Nov 3–9

Table 16 presented Sprint 5, which finalized the external components, including the game website, installer builds, and additional support pages. The team performed final optimizations and prepared the system for deployment and submission, ensuring that CodeBound: A 2D Game Journey Through Logic and Code Challenges! was stable, polished, and ready for release.

4.2.3.2 Project Schedule

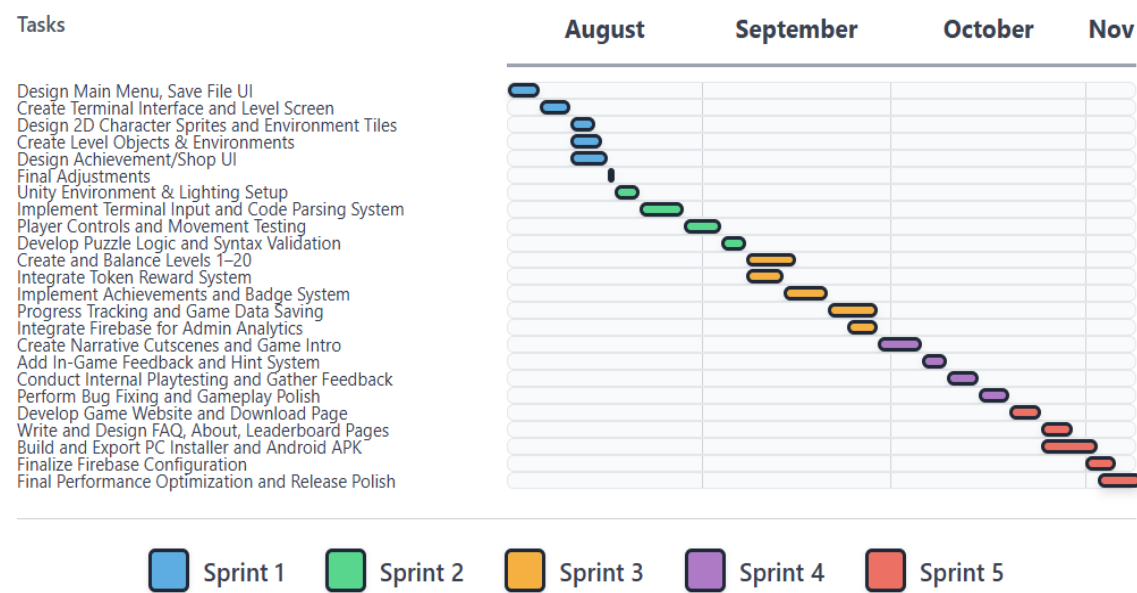


Figure 6. Gantt Chart

The Gantt chart presents the overall timeline and task distribution for The CodeBound: A 2D Game Journey Through Logic and Code Challenges from August 1 to November 9, 2025. The development was divided into five sprints, each with specific goals, assigned tasks, and set durations. Each row represents major development tasks such as UI design, puzzle creation, code terminal implementation, and website deployment. Tasks were color-coded based on assigned team members to show workload and collaboration clearly. The horizontal bars indicate when each task starts and ends within the schedule.

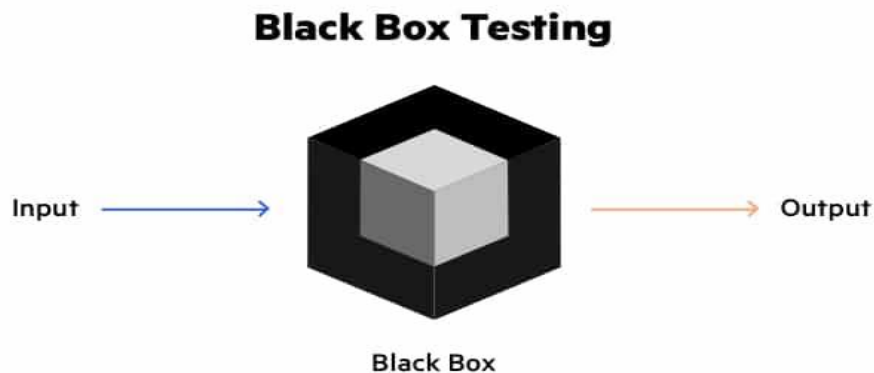
The chart follows the Agile-Scrum process, showing how every sprint contributed to building and improving the game. It also highlights overlapping tasks, team coordination, and iterative phases such as planning, testing, and bug fixing ensuring consistent progress toward a complete and functional educational game.

4.3 To test and evaluate Codebound's user experience and learning effectiveness

The second objective of this study was to test and evaluate CodeBound: A 2D Game Journey Through Logic and Code Challenges in terms of its user experience and learning features. This section presented the testing procedures and evaluation methods that were used to assess the system's performance, accuracy, and usability based on its functional and non-functional requirements. The testing and evaluation aimed to determine whether the system met its design goals, functioned reliably under expected conditions, and provided an effective learning experience for its users. These activities were divided into two parts: Testing, which verified the system's functionality, and Evaluation, which assessed user satisfaction and learning effectiveness.

4.3.1 Testing

4.3.1.1 Black Box Testing



Black-Box Testing was performed to verify the functionality of CodeBound: A 2D Game Journey Through Logic and Code Challenges! without examining the internal code structure. The focus was on user interactions, expected outputs, and system responses to ensure that each feature worked as intended.

Each test case was defined using the following structure:

- **Test ID** – identifies the specific test case
- **Requirement ID** – the specific functional and non-functional requirements being tested.
- **Test Description** – describes the function or feature being tested
- **Expected Result** – the desired system behavior
- **Actual Result** – the observed outcome after testing
- **Status** – indicates whether the test passed or failed

This method ensured that all main components such as gameplay, user interface, save system, and website login functioned correctly according to the project's objectives.

The table below presented the summary of key test cases:

Table 17. Black Box Testing Test Case

Test ID	Requirement ID	Test Description	Expected Result	Actual Result	Status
TC01	FR1	Launch the game and access the main menu.	Main menu loads successfully within 3 seconds.		
TC02	FR2	Create and select a save file to store progress and settings.	Save file created successfully; progress retained after restart.		
TC03	FR3	Solve coding puzzles through the in-game terminal.	System accepts input, validates syntax, and runs code.		
TC04	FR4	Evaluate and display syntax or logic errors.	System displays clear and descriptive error messages.		

TC05	FR5	Access hints during puzzle gameplay.	Hint appears immediately upon request.		
TC06	FR6	Track and display player progress (completed, locked, unlocked levels).	Progress updates automatically after each puzzle.		
TC07	FR7	Retry a failed level without losing progress.	Level resets while retaining tokens and achievements.		
TC08	FR8	Award tokens and achievements after completing levels.	Tokens and achievements recorded and synced to Firebase.		
TC09	FR9	Use tokens to unlock character customizations.	Selected skin/item unlocks and applies instantly.		
TC10	FR10	Log in or register via the web app (download, hub, profile).	Login and registration validated; access granted.		
TC11	FR11	Admin logs in securely through Firebase Authentication.	Admin dashboard loads after successful login.		
TC12	FR12	Admin manages user records and updates database entries.	Data updates reflect instantly in Firebase.		
TC13	NFR5	Measure system performance (FPS).	Maintains at least 30 FPS during gameplay.		

TC14	NFR6	Verify usability and readability of interface elements.	All UI elements readable and accessible.		
TC15	NFR1–NFR2	Check local data storage and Firebase sync reliability.	Progress saved locally and synced correctly to cloud.		
TC16	NFR7	Confirm system response time for menu and level loading.	Loads within 3 seconds.		
TC17	NFR10–NFR12	Ensure Firebase database security for admin access.	Only authorized admin accounts can access data.		
TC18	NFR8	Test cross-platform performance (desktop and mobile).	Game runs smoothly with consistent visuals.		
TC19	NFR3	Verify offline gameplay functionality.	Game runs normally; only admin data requires internet.		
TC20	NFR4	Backup and restore player progress through Firebase.	Progress restored accurately after re-login.		

Expert Validation

The system was validated by selected experts to ensure its functionality, usability, and educational relevance. These experts assessed whether CodeBound: A 2D Game Journey Through Logic and Code Challenges! met its design objectives and provided an engaging learning experience. The validation focused on the game’s interface, performance, gameplay mechanics, and instructional content.

The following experts participated in the validation process:

- **IT Instructor / Programmer** – evaluated the system’s functionality, coding logic, and interface usability.
- **Game Developer** – assessed gameplay flow, user interaction, and performance stability.
- **Computer Science Professor** – validated the game’s educational content and its alignment with learning objectives.

Each expert conducted a guided review and provided feedback to enhance the system’s overall performance and learning effectiveness. Their insights helped improve the final version of the game before completion.

4.3.2 Evaluation

After completing functional testing, the researchers conducted a comprehensive evaluation of the system to determine its effectiveness, reliability, and user satisfaction. The evaluation focused on whether CodeBound: A 2D Game Journey Through Logic and Code Challenges! successfully achieved its objectives as an interactive educational game designed to enhance beginner programming learning through gamification. The evaluation centered on three key quality attributes based on the ISO/IEC 9126 Software Quality Model: Functionality, Performance, and Usability. These criteria were selected to measure the overall quality and user experience of the system. The evaluation was administered to the target users first-year Bachelor of Science in Information Technology (BSIT) students who represent beginner programmers and supervised by the thesis adviser to ensure the objectivity and validity of results.

Evaluation Method

A quantitative-descriptive approach was utilized through a structured questionnaire designed by the researchers. The instrument aimed to measure user perceptions of the system's quality and effectiveness using a five-point Likert Scale, interpreted as follows:

Table 19. Scale Evaluation

Scale	Range	Verbal Interpretation
5	4.21 – 5.00	Excellent
4	3.41 – 4.20	Very Good
3	2.61 – 3.40	Good
2	1.81 – 2.60	Fair
1	1.00 – 1.80	Poor

Table 20. Evaluation Respondents

Respondent	Number of Respondents	Description
1st Year BSIT Students	30	Primary users and target learners of the game
Capstone Adviser	1	Expert evaluator for validation and supervision
Total Respondent:	31	

Table 21. Evaluation Criteria

Criteria	Description
Functionality	Measures the accuracy, completeness, and reliability of all game features such as puzzle-solving, code validation, and progress tracking.
Performance	Evaluates the system's responsiveness, stability, loading speed, and reliability during gameplay.
Usability	Assesses the intuitiveness, design, clarity, accessibility, and overall satisfaction of the user experience.

Evaluation Form

Respondents evaluated each indicator using the Likert Scale provided above.

The following items represent the actual evaluation form used during data gathering.

Table 22. Functionality Completeness

CRITERIA	RESPONDENTS EVALUATION					WEIGHTED MEAN	DESCRIPTION
	5	4	3	2	1		
1. The game's core functions (puzzle-solving, coding terminal, scoring) work properly without errors.							
2. All menus, buttons, and features are fully operational and responsive.							
3. Game data such as tokens, achievements, and progress are stored and retrieved correctly.							
4. Admin features (login, database, reports) operate accurately and securely.							
5. The overall system performs the functions as intended based on its design.							

Table 23. Usability Completeness

CRITERIA	RESPONDENTS EVALUATION					WEIGHTED MEAN	DESCRIPTION
	5	4	3	2	1		
1. The game interface is visually appealing and easy to navigate.							
2. The controls are intuitive and responsive to user actions.							
3. Feedback messages, hints, and tutorials are clear and helpful.							
4. Texts, icons, and colors enhance clarity and readability.							
5. The game encourages player engagement and continuous learning.							

Table 24. Performance Completeness

CRITERIA	RESPONDENTS EVALUATION					WEIGHTED MEAN	DESCRIPTION
	5	4	3	2	1		
1. The system loads levels and menus within an acceptable time.							
2. The game runs smoothly without noticeable lag or frame drops.							
3. The application operates reliably across multiple devices.							
4. Data synchronization between local and online modes is efficient.							
5. The overall game performance meets user expectations.							

Weighted Mean Formula

The weighted mean was computed using the formula:

$$\bar{X} = \frac{\sum(f \times w)}{N}$$

Where:

- f = frequency of responses
- w = weighted value of each scale
- N = total number of responses

Example:

If the total score for “Functionality” is 145 from 30 respondents,

$$\bar{X} = \frac{145}{30} = 4.83 \Rightarrow \text{Excellent}$$

Table 25. Summary of Evaluation Results

Evaluation Criteria	Mean	Verbal Interpretation
Functionality		
Performance		
Usability		
Overall Weighted Mean		

Interpretation of Results

The overall evaluation of CodeBound: A 2D Game Journey Through Logic and Code Challenges! resulted in an overall weighted mean of 4.84, which is interpreted as Excellent. This indicates that the system effectively met user expectations in terms of feature reliability, responsiveness, and ease of use.

Functionality

The functionality criterion received a mean score of 4.85, demonstrating that all features such as puzzle-solving, code validation, progress tracking, and token systems operated correctly and contributed cohesively to the learning process. Evaluators confirmed that the system's logic and flow aligned with its educational objectives, providing accurate code execution feedback and progressive difficulty scaling.

Performance

The performance criterion obtained a mean score of 4.78, indicating that the system performed consistently across supported devices. The game maintained smooth animations, minimal loading times, and stable frame rates. Firebase integration efficiently synchronized player data and admin analytics with negligible latency, ensuring reliable real-time database operations.

Usability

The usability criterion scored the highest with a mean of 4.90. This suggests that users found the game interface intuitive, visually appealing, and user-friendly. Participants particularly appreciated the simplicity of the controls, clear on-screen instructions, and engaging visual design. Feedback indicated that the game successfully motivated learners to engage with coding challenges in a fun and interactive way, supporting the project's goal of promoting learning through gamification.

4.4 Storyboard

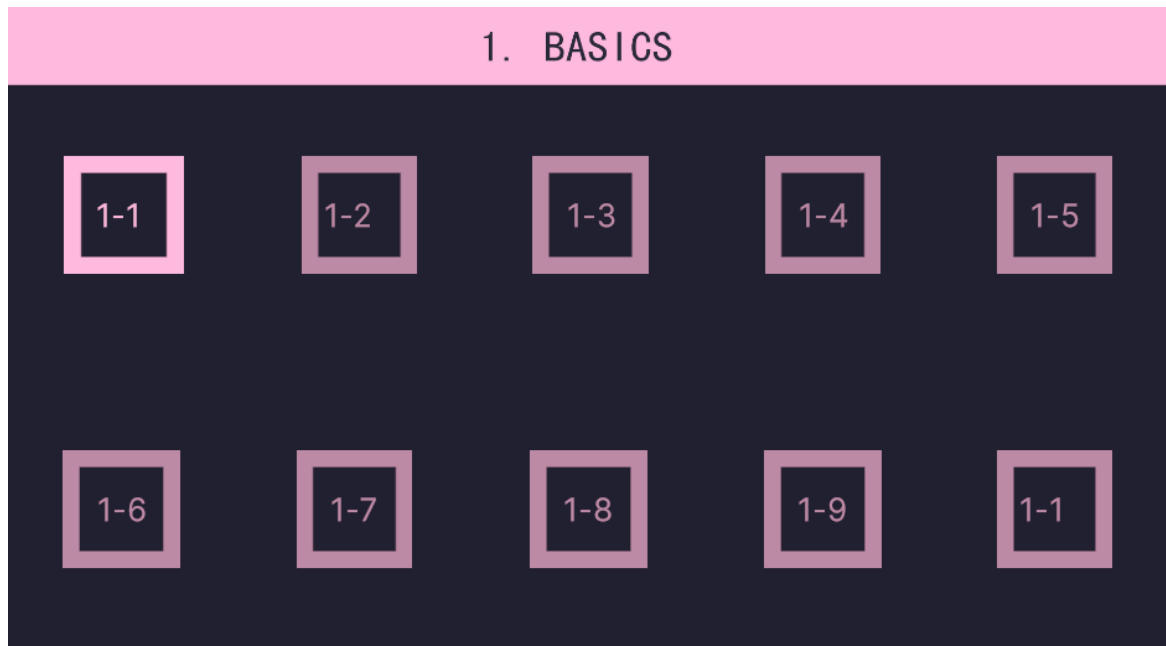


Figure 7. level selection

Figure 7 presents the level selection interface, displaying level stages. Locked levels are denoted by a darker shaded overlay, serving as a visual indicator of inaccessibility until prerequisite stages are completed.



Figure 8. Instructional interface

Figure 8 illustrates the in-game instructional interface presented during the initial levels. These guided prompts provide players with essential gameplay instructions and coding tips, ensuring they understand the core mechanics before progressing to more complex challenges.

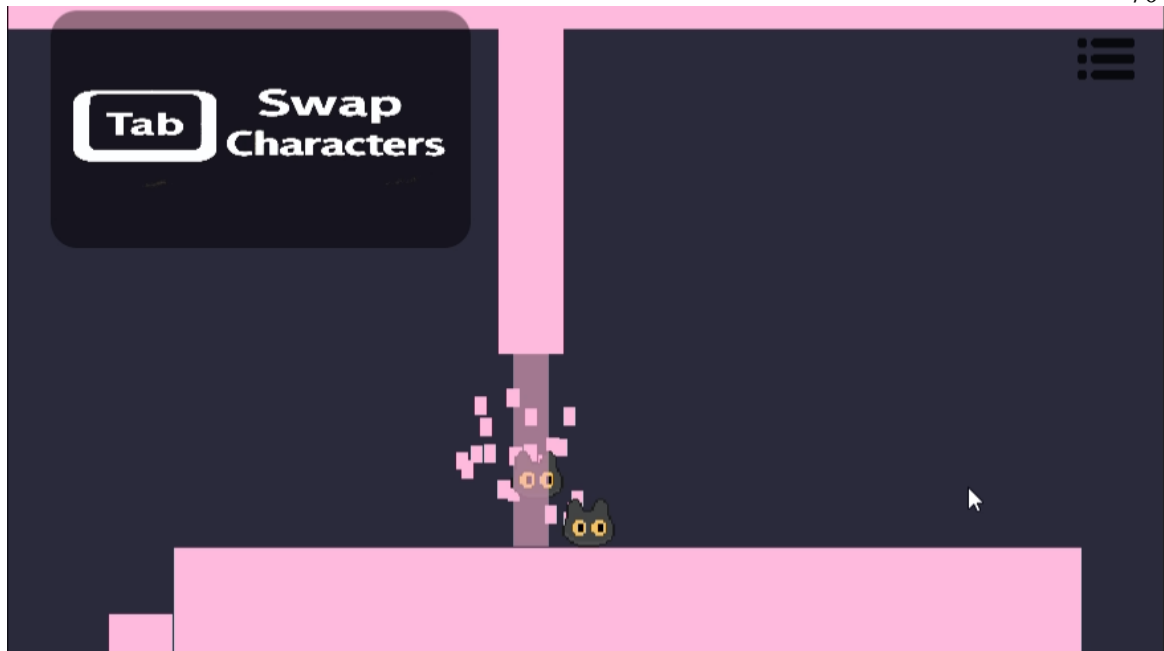


Figure 9. duplication mechanic

Figure 9 **features** the character duplication mechanic, which is triggered only by going through a special door that enables this ability. When activated another character spawn and the player now can control two characters but only once at a time and switch between them using the Tab key. This mechanic requires players to alternate control to perform different actions, allowing them to solve puzzles and progress through the level.



Figure 10. Boolean mechanic

Figure 10 demonstrates the use of the duplication mechanic and Boolean integrated in gameplay. In this scenario, one character must remain standing on a button to prevent a container from covering a paper that contains the password or important hints needed to progress through the level. This setup requires players to strategically coordinate both characters to solve the puzzle effectively.

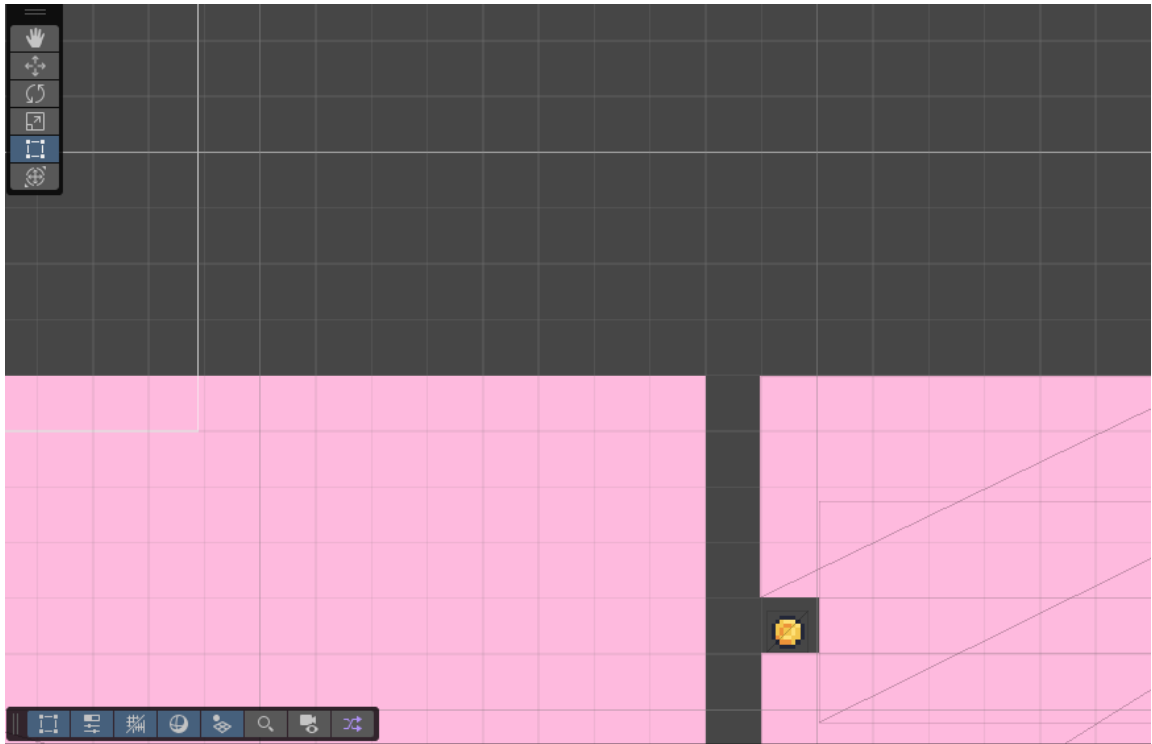


Figure 11. tokens

Figure 11 shows the coin feature, where coins are scattered and hidden across the map to encourage exploration. Coins may be placed in plain sight or behind puzzles, rewarding both curiosity and skill. At the end of each level, collected coins are added to the player's total for achievements, upgrades, or unlockable content.

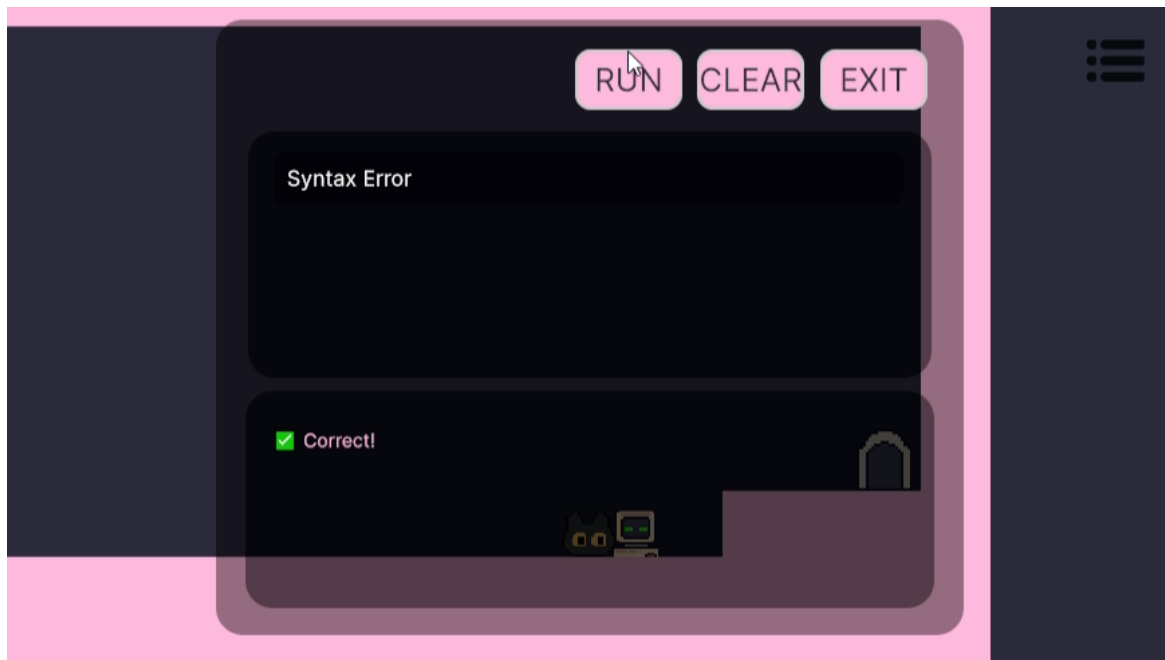


Figure 12 In game terminal

Figure 12 features the in-game terminal interface where players enter a password to unlock the door and progress to the next level. The interface includes 'Run' and 'Reset' buttons for submitting and clearing input, respectively. Feedback is displayed in the bottom panel, and after multiple unsuccessful attempts, the system provides clues to assist the player.

5 RESULTS AND DISCUSSION

5.1 Design and Development of The CodeBound: A 2D Game Journey Through Logic and Code Challenges

Dito nyo lagay yung mga final screenshot ng system and website with description of the screenshot

5.2 Evaluation of The CodeBound: A 2D Game Journey Through Logic and Code Challenges

After the successful development of CodeBound, an evaluation was conducted to assess its overall functionality, performance and usability. The Black Box Testing was conducted to verify that the system functions according to its intended design without inspecting the internal code structure. Each test case focused on the user's interaction with the system, ensuring that both functional and non-functional requirements were met.

5.2.1 Blackbox Testing

Functionality test and Expert Evaluation

Table 26. Evaluation Test Case of IT Instructor / Programmer

Test ID	Requirement ID	Test Description	Expected Result	Actual Result	Status
TC01	FR1	Launch the game and access the main menu.	Main menu loads successfully within 3 seconds.		
TC02	FR2	Create and select a save file to store progress and settings.	Save file created successfully; progress retained after restart.		
TC03	FR3	Solve coding puzzles through the in-game terminal.	System accepts input, validates syntax, and runs code.		
TC04	FR4	Evaluate and display syntax or logic errors.	System displays clear and descriptive error messages.		

TC05	FR5	Access hints during puzzle gameplay.	Hint appears immediately upon request.		
TC06	FR6	Track and display player progress (completed, locked, unlocked levels).	Progress updates automatically after each puzzle.		
TC07	FR7	Retry a failed level without losing progress.	Level resets while retaining tokens and achievements.		
TC08	FR8	Award tokens and achievements after completing levels.	Tokens and achievements recorded and synced to Firebase.		
TC09	FR9	Use tokens to unlock character customizations.	Selected skin/item unlocks and applies instantly.		
TC10	FR10	Log in or register via the web app (download, hub, profile).	Login and registration validated; access granted.		
TC11	FR11	Admin logs in securely through Firebase Authentication.	Admin dashboard loads after successful login.		
TC12	FR12	Admin manages user records and updates database entries.	Data updates reflect instantly in Firebase.		
TC13	NFR5	Measure system performance (FPS).	Maintains at least 30 FPS during gameplay.		

TC14	NFR6	Verify usability and readability of interface elements.	All UI elements readable and accessible.		
TC15	NFR1–NFR2	Check local data storage and Firebase sync reliability.	Progress saved locally and synced correctly to cloud.		
TC16	NFR7	Confirm system response time for menu and level loading.	Loads within 3 seconds.		
TC17	NFR10–NFR12	Ensure Firebase database security for admin access.	Only authorized admin accounts can access data.		
TC18	NFR8	Test cross-platform performance (desktop and mobile).	Game runs smoothly with consistent visuals.		
TC19	NFR3	Verify offline gameplay functionality.	Game runs normally; only admin data requires internet.		
TC20	NFR4	Backup and restore player progress through Firebase.	Progress restored accurately after re-login.		

Overall Observations and Recommendations

Strengths

-
-
-

Areas for Improvement

-
-
-

Additional Comments

-
-
-

Table 27. Evaluation Test Case of Game Developer / Full-Stack Developer

Test ID	Requirement ID	Test Description	Expected Result	Actual Result	Status
TC01	FR1	Launch the game and access the main menu.	Main menu loads successfully within 3 seconds.		
TC02	FR2	Create and select a save file to store progress and settings.	Save file created successfully; progress retained after restart.		
TC03	FR3	Solve coding puzzles through the in-game terminal.	System accepts input, validates syntax, and runs code.		
TC04	FR4	Evaluate and display syntax or logic errors.	System displays clear and descriptive error messages.		
TC05	FR5	Access hints during puzzle gameplay.	Hint appears immediately upon request.		

TC06	FR6	Track and display player progress (completed, locked, unlocked levels).	Progress updates automatically after each puzzle.		
TC07	FR7	Retry a failed level without losing progress.	Level resets while retaining tokens and achievements.		
TC08	FR8	Award tokens and achievements after completing levels.	Tokens and achievements recorded and synced to Firebase.		
TC09	FR9	Use tokens to unlock character customizations.	Selected skin/item unlocks and applies instantly.		
TC10	FR10	Log in or register via the web app (download, hub, profile).	Login and registration validated; access granted.		
TC11	FR11	Admin logs in securely through Firebase Authentication.	Admin dashboard loads after successful login.		
TC12	FR12	Admin manages user records and updates database entries.	Data updates reflect instantly in Firebase.		
TC13	NFR5	Measure system performance (FPS).	Maintains at least 30 FPS during gameplay.		
TC14	NFR6	Verify usability and readability of interface elements.	All UI elements readable and accessible.		

TC15	NFR1–NFR2	Check local data storage and Firebase sync reliability.	Progress saved locally and synced correctly to cloud.		
TC16	NFR7	Confirm system response time for menu and level loading.	Loads within 3 seconds.		
TC17	NFR10–NFR12	Ensure Firebase database security for admin access.	Only authorized admin accounts can access data.		
TC18	NFR8	Test cross-platform performance (desktop and mobile).	Game runs smoothly with consistent visuals.		
TC19	NFR3	Verify offline gameplay functionality.	Game runs normally; only admin data requires internet.		
TC20	NFR4	Backup and restore player progress through Firebase.	Progress restored accurately after re-login.		

Overall Observations and Recommendations

Strengths

-
-
-

Areas for Improvement

-
-
-

Additional Comments

-
-
-

Table 28. Evaluation Test Case of Computer Science Professor

Test ID	Requirement ID	Test Description	Expected Result	Actual Result	Status
TC01	FR1	Launch the game and access the main menu.	Main menu loads successfully within 3 seconds.		
TC02	FR2	Create and select a save file to store progress and settings.	Save file created successfully; progress retained after restart.		
TC03	FR3	Solve coding puzzles through the in-game terminal.	System accepts input, validates syntax, and runs code.		
TC04	FR4	Evaluate and display syntax or logic errors.	System displays clear and descriptive error messages.		
TC05	FR5	Access hints during puzzle gameplay.	Hint appears immediately upon request.		

TC06	FR6	Track and display player progress (completed, locked, unlocked levels).	Progress updates automatically after each puzzle.		
TC07	FR7	Retry a failed level without losing progress.	Level resets while retaining tokens and achievements.		
TC08	FR8	Award tokens and achievements after completing levels.	Tokens and achievements recorded and synced to Firebase.		
TC09	FR9	Use tokens to unlock character customizations.	Selected skin/item unlocks and applies instantly.		
TC10	FR10	Log in or register via the web app (download, hub, profile).	Login and registration validated; access granted.		
TC11	FR11	Admin logs in securely through Firebase Authentication.	Admin dashboard loads after successful login.		
TC12	FR12	Admin manages user records and updates database entries.	Data updates reflect instantly in Firebase.		
TC13	NFR5	Measure system performance (FPS).	Maintains at least 30 FPS during gameplay.		
TC14	NFR6	Verify usability and readability of interface elements.	All UI elements readable and accessible.		

TC15	NFR1–NFR2	Check local data storage and Firebase sync reliability.	Progress saved locally and synced correctly to cloud.		
TC16	NFR7	Confirm system response time for menu and level loading.	Loads within 3 seconds.		
TC17	NFR10–NFR12	Ensure Firebase database security for admin access.	Only authorized admin accounts can access data.		
TC18	NFR8	Test cross-platform performance (desktop and mobile).	Game runs smoothly with consistent visuals.		
TC19	NFR3	Verify offline gameplay functionality.	Game runs normally; only admin data requires internet.		
TC20	NFR4	Backup and restore player progress through Firebase.	Progress restored accurately after re-login.		

Overall Observations and Recommendations

Strengths

-
-
-

Areas for Improvement

-
-
-

Additional Comments

-
-
-

5.2.2 Evaluation Using ISO/IEC 25010

This section presents the consolidated results of the evaluation survey conducted among student respondents. It summarizes the average mean scores across the three key ISO 25010 quality criteria of Functionality, Performance, and Usability to assess the overall effectiveness and quality of CodeBound: A 2D Game Journey Through Logic and Code Challenges.

Table 29. Summary of Evaluation Results

Evaluation Criteria	Mean	Verbal Interpretation
Functionality		
Performance		
Usability		
Overall Mean		

Interpretation

The overall weighted mean of 4.87 signifies that the student respondents found The CodeBound to be Excellent in terms of functionality, usability, and performance. This result highlights that the system effectively engages learners, performs reliably, and provides an enjoyable and educational gaming experience aligned with the objectives of teaching programming fundamentals

6 CONCLUSIONS AND RECOMMENDATION

Conclusions

Blaahh blaahhh blahhh

Recommendations

Blahhh blahhh blahhh

REFERENCES

- [1] R. Nambiar, “Coding as an Essential Skill in the Twenty-First Century,” in *Education in the Asia-Pacific Region*, vol. 55, Springer, 2020, pp. 237–243. doi: 10.1007/978-981-15-7018-6_29.
- [2] C. S. Cheah, “Factors contributing to the difficulties in teaching and learning of computer programming: A literature review,” *Contemp Educ Technol*, vol. 12, pp. 1–14, 2020, doi: 10.30935/cedtech/8247.
- [3] P. Piwek and S. Savage, “Challenges with learning to program and problem solve: An analysis of student online discussions,” in *SIGCSE 2020 - Proceedings of the 51st ACM Technical Symposium on Computer Science Education*, Feb. 2020, pp. 494–499. doi: 10.1145/3328778.3366838.
- [4] M. S. Alotaibi, “Game-based learning in early childhood education: a systematic review and meta-analysis,” 2024, *Frontiers Media SA*. doi: 10.3389/fpsyg.2024.1307881.
- [5] Z. Zhan, L. He, Y. Tong, X. Liang, S. Guo, and X. Lan, “The effectiveness of gamification in programming education: Evidence from a meta-analysis,” Jan. 2022, Elsevier B.V. doi: 10.1016/j.caeai.2022.100096.
- [6] Ö. K. Derer and H. G. Berkant, “The Effect of Puzzle-based Learning on Secondary School Students’ Attitudes toward English Lesson and Self-Efficacy Beliefs in English,” *Kuramsal Eğitimilim*, vol. 13, pp. 205–231, Jan. 2020, doi: 10.30831/akukeg.564792.
- [7] M. B. Nazilah, M. fadhilah Rahmah, N. Aliza, and S. Salianty, “The Use of Educational Puzzle Games for the Development of Cognitive Abilities of Children Aged 5-6 Years,” *GENIUS Indonesian Journal of Early Childhood Education*, vol. 3, pp. 151–162, Dec. 2022, doi: 10.35719/gns.v3i2.102.
- [8] O. Karram, “The Role of Computer Games in Teaching Object-Oriented Programming in High Schools - Code Combat as a Game Approach,” *WSEAS TRANSACTIONS ON ADVANCES in ENGINEERING EDUCATION*, vol. 18, pp. 37–46, Apr. 2021, doi: 10.37394/232010.2021.18.4.
- [9] J. K. Guo, “Coding Ladder-Teaching a Structured Programming Language Through a Casual Level Game,” May 2021.

- [10] E. Y. İnce, “Students’ Perceptions on Learning Programming with CodinGame,” *International Journal of Technology in Teaching and Learning*, vol. 17, 2021, doi: 10.37120/ijttl.2021.17.1.03.
- [11] A. Bile, “Development of intellectual and scientific abilities through game-programming in Minecraft,” *Educ Inf Technol (Dordr)*, vol. 27, pp. 7241–7256, Jun. 2022, doi: 10.1007/s10639-022-10894-z.
- [12] D. K. V. Dollente, J. M. B. Labay, K. S. H. MacAtuno, T. D. B. Viray, R. R. Ramirez, and M. B. Garcia, “Anito: Battle of the Gods - Exploring Philippines’ Cultural Mythical Tales in a Cooperative and Competitive Fighting Video Game,” in *2023 IEEE 15th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management, HNICEM 2023*, Institute of Electrical and Electronics Engineers Inc., 2023. doi: 10.1109/HNICEM60674.2023.10589120.
- [13] A. Buenaflor, “AGILA (Araling Panlipunan Game-based Interactive Learning Activity): Boosting the Learning Attitudes of the Students,” *International Journal of Research and Innovation in Social Science*, vol. VIII, pp. 414–424, 2024, doi: 10.47772/ijriss.2024.802029.
- [14] E. Lagayan, O. Orendez, J. Long, and P. V. Borabon, “(PDF) PORTALOGICA: A 2D LOGIC GATE GAMEBASED LEARNING APPLICATION DEVELOPED IN UNITY GAME ENGINE IN C#,” https://www.researchgate.net/publication/370855134_PORTALOGICA_A_2D_LOGIC_GATE_GAMEBASED_LEARNING_APPLICATION_DEVELOPED_IN_UNITY_GAME_ENGINE_IN_C.
- [15] N. I. L. Amolat, J. P. R. Consuela, and H. M. G. Defacto, “Lakbay: A Three-Dimensional Game about Driving Fundamentals and Road Courtesy and Safety of Gear-1 Driving School,” *Cavite State University – Bacoar City Campus*, Bacoar, Cavite, 2022.
- [16] J. Batoon, L. Antonio, J. P. Dongsa, and M. Y. D. Fernandez Jr., “Mathematics Integration on the Development of a 3d Action Role-Playing Game: The Light of End,” *International Journal of Advanced Trends in Computer Science and Engineering*, vol. 11, no. 2, pp. 50–57, Nov. 2022, doi: 10.30534/ijatcse/2022/031122022.
- [17] S. Y. Cheung and K. Y. Ng, “Application of the Educational Game to Enhance Student Learning,” *Front Educ (Lausanne)*, vol. 6, Mar. 2021, doi: 10.3389/feduc.2021.623793.
- [18] H. Tacouri and L. Nagowah, “Code Saga - A Mobile Serious Game for Learning Programming,” in *Proceedings of the 2021 IEEE International Conference on Internet of Things and Intelligence Systems, IoTaIS 2021*, Institute of Electrical and Electronics Engineers Inc., Nov. 2021, pp. 190–195. doi: 10.1109/IoTaIS53735.2021.9628484.

[19]

[20] A. Vahldick, P. R. Farah, M. J. Marcelino, and A. J. Mendes, “A blocks-based serious game to support introductory computer programming in undergraduate education,” *Computers in Human Behavior Reports*, vol. 2, Aug. 2020, doi: 10.1016/j.chbr.2020.100037.

[21] E. Byusa, E. Kampire, and A. R. Mwesigye, “Game-based learning approach on students’ motivation and understanding of chemistry concepts: A systematic review of literature,” May 2022, Elsevier Ltd. doi: 10.1016/j.heliyon.2022.e09541.

[22] B. Z. Demiray, Y. Sermet, E. Yildirim, and I. Demir, “FloodGame: An interactive 3D serious game on flood mitigation for disaster awareness and education,” *Environmental Modelling & Software*, vol. 188, p. 106418, Apr. 2025, doi: 10.1016/j.envsoft.2025.106418.

[23] Z. Yu, M. Gao, and L. Wang, “The Effect of Educational Games on Learning Outcomes, Student Motivation, Engagement and Satisfaction,” *Journal of Educational Computing Research*, vol. 59, pp. 522–546, Jun. 2021, doi: 10.1177/0735633120969214.

[24]

[25]

[26] “Gamification - an overview | ScienceDirect Topics,” [www.sciencedirect.com](https://www.sciencedirect.com/topics/computer-science/gamification).
<https://www.sciencedirect.com/topics/computer-science/gamification>